A Project Report

On

Agentic AI Procurement Solution

Submitted in partial fulfilment of the requirements of

University of Mumbai

For the Degree of

Bachelor of Engineering

in

COMPUTER ENGINEERING

Submitted by

More Anjali Dinesh
Sankaye Vaidehi Jagdeep
Sawant Niraj Ravindra
Shete Anooj

Supervised by

Dr. Jyoti More

&

Mr. Alok
(External guide)



Department of Computer Engineering
Fr. Conceicao Rodrigues Institute of Technology
An Autonomous Institute & Permanently Affiliated to University of Mumbai
Sector 9A, Vashi, Navi Mumbai - 400703

UNIVERSITY OF MUMBAI
2026–2027

APPROVAL SHEET

This is to certify that the project entitled

AGENTIC AI PROCUREMENT SOLUTION

Submitted by

More Anjali Dinesh	**1023207**
Sankaye Vaidehi Jagdeep	**1023239**
Sawant Niraj Ravindra	**1023240**
Shete Anooj	**1023250**

Supervisor / Project Guide	**Project Coordinator**
Internal Examiner	**External Examiner**
Head of Department	**Principal**

COLLEGE SEAL

Date : _____

Place : _____

Declaration

We declare that this written submission for B.E. Declaration entitled "**Agentic AI Procurement Solution**" represents our ideas in our own words and where others' ideas or words have been included. We have adequately cited and referenced the sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented, fabricated, or falsified any ideas/data/fact/source in our submission. We understand that any violation of the above will cause disciplinary action by the institute and also evoke penal action from the sources which have thus not been properly cited or from whom permission has not been taken when needed.

Project Group Members:

1. More Anjali Dinesh, 1023207

_____

2. Sankaye Vaidehi Jagdeep, 1023239

_____

3. Sawant Niraj Ravindra, 1023240

_____

4. Shete Anooj, 1023250

_____

Abstract

IT procurement—buying, licensing, renewing and vetting hardware, software and SaaS—is still largely run through e-mail, spreadsheets and disconnected tools. The consequences are slow approvals, duplicate payments, idle licences, missed renewal deadlines and vendor risk that is assessed once and then forgotten. Techniques exist for each stage, but they are usually applied in isolation.

This report presents an IT Procurement Intelligence Platform built as four cooperating agents: Document and Vendor Intelligence, Approval and Inventory Intelligence, Contract and Risk Analysis, and a Platform agent for authentication, business rules and notifications. Each agent is an independent service that reacts to events on a Kafka-compatible bus, decides with rules or trained models, and publishes its result through a transactional outbox; long-running steps are durable Temporal workflows, and uncertain or sensitive decisions are handed to a person. Trained models are adopted only where they beat a simple baseline on held-out data.

On synthetic data, a TF-IDF classifier reached a macro-F1 of 1.0 against 0.912 for keyword rules; a Legal-BERT model reached F1 scores of 0.85, 0.80 and 0.91 on three clause types of the CUAD test split, against 0.50, 0.19 and 0.02 for rules; an Isolation Forest separated anomalous from normal licence usage with a ROC-AUC of 0.954; and a Random Forest risk model reached a ROC-AUC of 0.715 against a ceiling of 0.739 set by its label noise. Field extraction was complete on text PDFs but reached only 49% on image documents. On 3,000 generated invoices, the current confidence rule accepted 41% of invoices without review, 6.8% of them wrongly; adding purchase-order and vendor-master evidence raised the ranking quality of a calibrated gate from an AUROC of 0.74 to 0.95, but no gate could be certified at an error budget of 5% or less, chiefly because the duplicate check misses resubmitted invoices with a new number. The implementation passes 844 service unit tests and end-to-end tests through the gateway. Because the data is synthetic, these figures compare methods; they do not predict performance on an organisation's own documents.

Keywords: IT procurement, event-driven architecture, document intelligence, human-in-the-loop, selective automation, vendor risk, anomaly detection, contract analysis.

Contents

Abstract iii

List of Figures vii

List of Tables viii

List of Listings ix

1 Introduction 1
- 1.1 Background . . . 2
- 1.2 Motivation . . . 2
- 1.3 Aim and Objectives . . . 2
  - 1.3.1 Aim . . . 2
  - 1.3.2 Objectives . . . 2
- 1.4 Problem Statement . . . 3
- 1.5 Scope . . . 3
- 1.6 Report Outline . . . 3

2 Study of the System 4
- 2.1 About the Technique . . . 5
- 2.2 Various Available Techniques . . . 6
  - 2.2.1 Document Intelligence . . . 6
  - 2.2.2 Document Classification . . . 6
  - 2.2.3 Vendor Deduplication and Identity Verification . . . 6
  - 2.2.4 Workflow Orchestration and Concurrency Control . . . 7
  - 2.2.5 Usage Anomaly Detection and Explainability . . . 7
  - 2.2.6 Contract Analysis and Clause Extraction . . . 7
  - 2.2.7 Vendor Risk Scoring and Drift Monitoring . . . 7
  - 2.2.8 Multi-Agent and Agentic Architectures . . . 8
- 2.3 Related Works . . . 8
- 2.4 Commercial Procurement Platforms . . . 10
- 2.5 Literature Summary and Research Gap . . . 11

3 Proposed System 13
- 3.1 Problem Statement . . . 14
- 3.2 Scope . . . 14
- 3.3 Scope Design . . . 15
- 3.4 Proposed System . . . 15
  - 3.4.1 Overview and Architecture . . . 15
  - 3.4.2 The Four Agents . . . 16

3.4.3 Document Processing Pipeline . . . 18
3.4.4 Inter-Agent Communication and Workflow . . . 18
3.4.5 Use Case View . . . 20
3.4.6 Key Design Principles . . . 21
3.4.7 Technology Stack . . . 21

4 Design of the System **23**
4.1 Requirement Engineering . . . 24
4.1.1 Requirement Elicitation . . . 24
4.1.2 Software Lifecycle Model . . . 24
4.1.3 Requirement Analysis . . . 25
4.1.3.1 Functional Requirements . . . 25
4.1.3.2 Non-Functional Requirements . . . 27
4.1.4 UML Diagrams and Data Flow Diagrams . . . 27
4.1.4.1 Use Case Diagram . . . 27
4.1.4.2 Data Flow Diagram – Level 0 . . . 28
4.1.4.3 Data Flow Diagram – Level 1 . . . 28
4.1.4.4 Sequence Diagram . . . 29
4.1.4.5 Activity Diagram: Document Processing . . . 29
4.1.4.6 Purchase-Request State Machine . . . 29
4.1.4.7 Entity Relationship Diagram . . . 29
4.1.5 Cost Analysis . . . 33
4.1.6 Hardware and Software Requirements . . . 33
4.2 System Architecture . . . 33
4.2.1 UI/UX Design . . . 35
4.2.2 Block Diagram . . . 35
4.2.3 Main Data Paths . . . 35

5 Result and Discussion **38**
5.1 Screenshots of the System . . . 39
5.2 Sample Code (Main Logic) . . . 39
5.3 Testing . . . 43
5.3.1 Unit Testing . . . 43
5.3.2 Integration Testing . . . 44
5.3.3 Black-box Testing . . . 45
5.3.4 Security and Dependency Checks . . . 45
5.4 Evaluation of the Models . . . 45
5.4.1 Document Classification . . . 45
5.4.2 Field Extraction . . . 47
5.4.3 Clause Extraction . . . 47
5.4.4 Licence-Usage Anomaly Detection . . . 48
5.4.5 Vendor Risk Scoring . . . 49
5.4.6 Calibrating the Document Review Gate . . . 49
5.5 Discussion . . . 51

6 Conclusion & Future Scope **52**

References **55**

Acknowledgement **57**

Appendix A: Timeline Chart **58**

Appendix B: Sustainable Development Goal (SDG) Mapping **60**

Appendix C: Complex Engineering Problem Identification and Evaluation **61**

Appendix D: Project Outcomes **63**

List of Figures

3.1 Overall architecture of the proposed system 16
3.2 Document processing pipeline of the Document and Vendor Intelligence agent 18
3.3 Main event flows between the agents 19
3.4 Use case diagram of the IT procurement platform 21

4.1 Iterative and incremental development cycle 25
4.2 Level 0 data flow diagram (context diagram) 28
4.3 Level 1 data flow diagram 28
4.4 Sequence diagram of the purchase-request journey 30
4.5 Activity diagram of document processing 31
4.6 State machine of a purchase request 32
4.7 Entity relationship diagram of the main entities 32
4.8 Block diagram of the deployed system 36

5.1 Request page with the procurement lifecycle 39
5.2 Document review page 40
5.3 Vendor page with risk score and factors 40
5.4 Licence page with usage trend and explanation 41
5.5 Signed contract page 41
5.6 Control Panel with the live Kafka event log 42
5.7 Clause extraction F1 on the CUAD test split 48
5.8 Licence-anomaly detection on simulated test licences 48
5.9 Incident rate per risk band 49

A.1 Project timeline 59

List of Tables

2.1 Traditional procurement practices and their shortcomings 5
2.2 Comparison of related works 9
2.3 Comparison of related works (continued) 9
2.4 Comparison with five commercial procurement platforms 10
2.5 Techniques reviewed and their use in the proposed system 12

3.1 Problems in the current procurement process and their impact 14
3.2 Summary of the four agents in the proposed system 16
3.3 Event topics exchanged between the agents 19
3.4 Technology stack of the proposed system 22

4.1 Functional requirements 25
4.2 Non-functional requirements 27
4.3 Level 1 DFD processes 29
4.4 Indicative cost analysis 33
4.5 Hardware requirements 34
4.6 Software requirements 34
4.7 Principal screens and their purposes 36
4.8 Architectural layers 37

5.1 Unit tests per service (run on 19 October 2026) 44
5.2 End-to-end test suites 45
5.3 Black-box test cases (run through the gateway, 17–19 October 2026) 46
5.4 Document classification on synthetic test sets (accuracy and macro-F1) . . 47
5.5 Field-extraction accuracy by field (share of documents with the field correct) 47
5.6 Review-gate systems on 3,000 synthetic invoices 50

List of Listings

5.1 Confidence scoring and the review gate 42
5.2 Stock reservation under a Redis lock 42
5.3 HMAC verification and replay protection 43

Chapter 1

Introduction

1.1 Background

IT procurement covers the acquisition and management of hardware, software licences, SaaS subscriptions and related services. A single purchase passes through several stages: a vendor's documents are received and checked, the request is approved, stock or licences are allocated, a contract is signed and renewed, and the vendor's risk is assessed. In many organisations these stages live in different places—e-mail, spreadsheets, scanned invoices and separate systems—so no one has a current view of a purchase from request to payment.

1.2 Motivation

Manual procurement is slow, and some of its errors are expensive. Retyped invoices introduce mistakes; the same invoice can be paid twice; licences remain paid for long after their users have left; contracts renew silently because a notice period was missed; and a request to change a supplier's bank details, if accepted without independent verification, diverts a payment to a fraudster. These failures share a cause: each stage is handled on its own, and decisions are taken without the evidence held by the other stages. A system that automates the routine steps, links the stages through shared records, and asks a person only when a decision is uncertain or sensitive would address that cause directly.

1.3 Aim and Objectives

1.3.1 Aim

To develop an IT Procurement Intelligence Platform that automates the main activities of the IT procurement lifecycle while keeping critical decisions under human control.

1.3.2 Objectives

1. Extract the key fields of purchase orders, invoices and quotations, with a confidence score for each result.
2. Match vendors against existing records, validate their tax and bank identifiers, and detect duplicate vendors and invoices.
3. Route purchase requests through spend-tier approval chains with deadlines, escalation and stock reservation.
4. Detect under-used software licences and propose their reclamation under human control.
5. Generate and sign contracts, extract their renewal terms and issue renewal reminders.
6. Score vendor risk with an explainable model and monitor the model for drift.
7. Provide authentication, role-based access, run-time business rules, notifications, monitoring and an audit trail.

8. Measure each model against a simple baseline, and measure how often the document review gate admits an error.

1.4 Problem Statement

Procurement data and decisions are fragmented across tools, so routine checks—whether an invoice duplicates an earlier one, whether its vendor and bank details are genuine, whether a licence is still used, whether a contract is about to renew—take manual effort and are often skipped. Automating these checks naively creates a different risk: a system that acts on extracted data without a measure of confidence, or without a person in the loop for sensitive changes, makes wrong procurement decisions at scale. The problem is therefore to automate the procurement lifecycle end to end while making every automated decision explainable, auditable and, where it is uncertain, subject to human review. Chapter 3 states the problem in detail.

1.5 Scope

The platform covers the path from an incoming document or purchase request to a signed contract, receipt of the goods and the matching of the vendor's invoice, together with licence reclamation, vendor risk scoring and the shared platform services. It is a decision-support and workflow system: payment execution, ERP integration and production deployment are outside its scope. Section 3.2 defines the scope precisely.

1.6 Report Outline

Chapter 2 reviews the techniques behind each part of the system and compares related work. Chapter 3 states the problem and describes the proposed system: its architecture, agents, document pipeline and event flow. Chapter 4 covers requirements, development approach, design models, resource requirements and system architecture. Chapter 5 presents the implemented system, its testing and the evaluation of its models and its review gate. Chapter 6 summarises the outcomes, limitations and future work.

Chapter 2

Study of the System

Procurement documents—purchase orders, invoices and quotations—arrive by e-mail, through vendor portals or as scans, rarely in the same layout twice, and are often still keyed in by hand. Contracts are followed up in spreadsheets, vendors are vetted once at onboarding, and licences are checked with an inactivity rule. The result is data-entry errors, duplicate payments, unverified vendors, silent renewals and idle licences, as well as exposure to Business Email Compromise (BEC), in which an attacker posing as a supplier has its bank details changed because nobody verifies the request independently.

Each agent of the proposed platform draws on a different set of methods, so this chapter reviews several techniques rather than one. It contrasts traditional practice with automated alternatives, reviews the techniques for each sub-problem together with the choice made in this project, compares ten related works and five commercial procurement platforms, and identifies the gap the project addresses.

2.1 About the Technique

Traditional procurement processing rests on four habits. The first is manual or template-based data entry: staff key in invoice and purchase-order details, or an OCR template is built for each vendor's layout. The second is static approval rules, where a request follows a fixed chain with no deadline, no escalation and no awareness of remaining stock. The third is periodic vendor review, which in practice often means one questionnaire at onboarding. The fourth is the flat-threshold licence audit, in which a seat is flagged when nobody has logged in for a fixed number of days. These habits work at small volumes but break down as an organisation grows; Table 2.1 summarises where each struggles.

Table 2.1: Traditional procurement practices and their shortcomings

Area	**Traditional approach**	**Shortcoming**
Data capture	Manual entry and per-vendor templates	Slow and error-prone; templates break whenever a vendor changes its layout.
Vendor verification	Manual lookup and e-mail confirmation	Vulnerable to BEC fraud; duplicate vendor records accumulate.
Approvals	E-mail chains and fixed rules	No SLA timers or escalation; overlapping requests can claim the same stock.
Licence management	Fixed inactivity threshold	Ignores usage patterns and wrongly flags seasonal or weekend dips.
Contract tracking	Calendar reminders	Notice periods are missed, resulting in unintended automatic renewals.
Risk assessment	One-time questionnaire	Becomes outdated; misses later events such as a cancelled tax registration or a sanctions listing.

These gaps motivate an automated, event-driven approach, but not one that hands every decision to a model. Pretrained components handle perception-heavy work such as reading page layouts and contract language; small models are trained where a task is specific to this project (document classification, licence anomalies and vendor risk), and each trained model is kept only if it beats a simple baseline on held-out data; and plain rules remain wherever certainty and auditability matter more than flexibility, such as spend-tier routing, sanctions overrides and dual control of bank details.

2.2 Various Available Techniques

2.2.1 Document Intelligence

Template and regular-expression approaches are fast and predictable but brittle when vendors change layouts. OCR engines such as Tesseract [1] and PaddleOCR [2] recover text from scanned pages but lose table structure. Layout-aware parsers such as Docling [3] recover reading order and tables, although they are not tuned to invoices. Multimodal transformers of the LayoutLM family [4, 5] give the best results on forms and invoices but need labelled data and more compute.

Choice in this project. PDFs with a usable text layer are read directly with pdfplumber; scanned PDFs go through Docling and its bundled OCR engine; images are read by PaddleOCR after deskewing and limited upscaling. PaddleOCR's native engine aborted the host process on some platform and version combinations, so it runs in an isolated subprocess with a time limit rather than in the worker itself. Fields are then extracted with regular expressions. When the LayoutLMv3 weights are installed, the model reads every document a second time as an independent cross-check (a disagreement forces human review) and replaces the OCR reading of an image whose OCR quality falls below 0.5.

2.2.2 Document Classification

TF-IDF features with logistic regression are light, fast to train and easy to inspect, which suits a small number of document classes. Transformer classifiers such as BERT have a higher ceiling but need more data and compute than this task justifies. A weighted keyword classifier, which scores title phrases such as "tax invoice" or "purchase order", needs no training data but depends on the vocabulary it is given.

Choice in this project. A TF-IDF (character 2–5-grams) and logistic-regression classifier labels each document as an invoice, purchase order, quotation or *other* paperwork. It was trained and evaluated on a corpus of 1,260 generated documents that includes OCR-noise and missing-title variants, and was kept only because it beat the keyword classifier on validation data and on document kinds held out of training (Section 5.4). The keyword classifier remains the fallback whenever the model is less than 60% confident.

2.2.3 Vendor Deduplication and Identity Verification

Fuzzy name matching (rapidfuzz) tolerates spelling differences and corporate suffixes but can produce false matches on its own. Official identifiers are more reliable: an Indian GSTIN carries a check digit that can be validated offline, and an IFSC code can be looked up in a public bank directory, while live registry lookups are limited by API quotas.

Choice in this project. Extracted vendor names are normalised and matched with a fuzzy ratio threshold of 88. Vendor onboarding validates GSTIN format and check digit and looks IFSC codes up in a public directory, flags names 70–87% similar to an existing vendor as possible impostors, and never writes bank details directly: they become a pending change that a second finance or admin user must verify. Every upload is scanned with ClamAV, which is signature-based and therefore fast but offers no protection against zero-day threats.

2.2.4 Workflow Orchestration and Concurrency Control

Cron jobs and scripts are simple but not durable and handle deadlines poorly. Temporal offers durable timers, signals and replay, at the cost of a steeper learning curve. Redis or database locks stop two requests from claiming the same stock, but need careful failure handling. Kafka-compatible brokers [6] provide decoupled, replayable events between services, provided that consumers are idempotent; and a transactional outbox [7] ensures that an event is published if and only if the database change that caused it commits.

Choice in this project. Temporal runs the approval, reclaim and renewal workflows; a short-lived Redis lock guards each stock reservation; Redpanda is the event bus; and events are written through a transactional outbox and consumed with retries and a dead-letter table.

2.2.5 Usage Anomaly Detection and Explainability

A flat inactivity threshold is transparent but flags seasonal and weekend dips by mistake. Isolation Forest [8] is unsupervised and efficient on tabular data, though it is sensitive to its contamination setting and gives no explanation of its own; SHAP [9] supplies per-feature contributions at some extra cost.

Choice in this project. An Isolation Forest scores each licence from seven scale-free usage features (utilisation over 7, 30 and 90 days, user retention over 7 and 30 days, days since the last login and weekend share), and SHAP values explain which features pushed a licence towards "anomalous". The decision thresholds were chosen on a validation set, and a flagged licence becomes a reclaim request that the licence holder can contest during a seven-day grace period before it reaches an approver.

2.2.6 Contract Analysis and Clause Extraction

Keyword search misses paraphrased clauses. Legal-BERT [10] fine-tuned on CUAD [11] handles legal wording better and answers one question per clause, but it was trained mostly on US commercial contracts and is slow on a CPU. FinBERT [12] scores financial sentiment, which could serve as an adverse-media signal, although sentiment is only a rough proxy for risk.

Choice in this project. A publicly released Legal-BERT checkpoint fine-tuned on CUAD's training split [13] extracts the renewal term, the notice period and the expiration date; regular-expression rules are the fallback when the model is not installed or finds too little. The model was kept as the primary method because it beat the rules on CUAD's test split and read every value of the project's own templates correctly (Section 5.4). FinBERT was studied but not implemented.

2.2.7 Vendor Risk Scoring and Drift Monitoring

Static questionnaires are snapshots, whereas a learned classifier can combine several signals into one score. Random Forests [14] are robust on small tabular datasets and provide feature importances for explanation. Some facts, however, should not be weighed against others: a cancelled tax registration or a sanctions listing should decide the outcome on its own. The Population Stability Index (PSI) [15] compares the distribution of recent scores with the training distribution and signals model drift.

Choice in this project. Hard overrides are applied first (a cancelled or suspended GSTIN, or a potential sanctions match against the OpenSanctions consolidated list [16],

forces a High band); otherwise a Random Forest on six vendor features produces a probability that is mapped to Low, Medium or High. Missing signals are left out rather than counted against the vendor. A weekly PSI check flags drift but never retrains the model, so a person decides what to do with the signal.

2.2.8 Multi-Agent and Agentic Architectures

Multi-agent systems split a workflow among specialised components with clear ownership, at the cost of integration effort. Large-language-model (LLM) frameworks such as AutoGen [17] make agents cooperate through free-form conversation, which is flexible but hard to audit.

Choice in this project. In this report an *agent* is an independent service that observes events, decides with rules or trained models within the authority given to it by business rules, acts by changing its own data and publishing new events, and hands a decision to a person when its confidence is low or the action is sensitive. Coordination is through a shared event contract rather than conversation, which keeps every decision deterministic and auditable. An LLM is used in one place only: a conversational assistant that answers users' questions through read-only tool calls and makes no procurement decisions.

2.3 Related Works

Ten published works were examined, each covering one building block of the platform. They are grouped below by the problem they address and compared in Tables 2.2 and 2.3.

Document understanding. LayoutLM [4] was among the first models to learn from text together with its two-dimensional position on the page, and improved results on forms, receipts and invoices considerably. LayoutLMv3 [5] added image information through unified text and image masking and dropped the CNN backbone, giving a simpler and stronger model. On the tooling side, Docling [3] rebuilds reading order and table structure locally, and PaddleOCR 3.0 [2] provides multilingual OCR with structure analysis, although it runs best on a GPU.

Contract and risk language. CUAD [11] provides 510 expert-annotated contracts covering 41 clause categories and has become the standard benchmark for clause extraction. LEGAL-BERT [10] showed that pretraining on legal text clearly outperforms generic BERT on legal tasks. For the financial side of vendor risk, FinBERT [12] contributes sentiment analysis tuned to financial language.

Anomaly detection and explanation. Isolation Forest [8] finds unusual records by how quickly random splits isolate them, so it needs no labelled examples. SHAP [9] explains a single prediction by how much each feature pushed the score up or down.

Agent frameworks. AutoGen [17] represents the LLM-driven approach in which agents cooperate by conversation. It is included as a contrast to the event-driven design chosen here.

Taken together, the works show a clear pattern. The document-AI papers concentrate on how accurately fields can be read and say little about what should happen next, such as matching the vendor, catching duplicates or holding a suspicious bank-detail change

Table 2.2: Comparison of related works

Reference	Contribution	Advantages	Limitations	Role in this project
Xu et al., LayoutLM (KDD 2020) [4]	Joint pretraining of text and 2-D layout	Large gains on forms, receipts and invoices	No image features in pretraining; needs fine-tuning	Studied; basis of the LayoutLMv3 cross-check
Huang et al., LayoutLMv3 (ACM MM 2022) [5]	Unified text–image masking, no CNN backbone	Strong results; simpler architecture	Compute-heavy; needs labels for new document types	Implemented as an optional cross-check and image fallback
Auer et al., Docling (2024) [3]	Local layout, reading-order and table recovery	Fast, local, strong on tables	Not tuned to invoices; weak on poor scans	Implemented for scanned PDFs
Cui et al., PaddleOCR 3.0 (2025) [2]	OCR and structure-analysis toolkit	Accurate, multilingual	GPU preferred; native crashes on some platforms	Implemented for images, isolated in a subprocess
Hendrycks et al., CUAD (NeurIPS 2021) [11]	510 contracts, 41 clause categories	Realistic, expert-annotated benchmark	English, mostly US contracts	Training data of the clause model; its test split is used for evaluation

Table 2.3: Comparison of related works (continued)

Reference	Contribution	Advantages	Limitations	Role in this project
Chalkidis et al., LEGAL-BERT (EMNLP 2020) [10]	BERT pretrained on legal corpora	Beats generic BERT on legal tasks	Smaller corpus; mostly English	Implemented: CUAD-fine-tuned checkpoint extracts renewal clauses
Araci, FinBERT (2019) [12]	BERT further pretrained on financial text	Strong financial sentiment	Sentiment is only a proxy for risk	Studied; adverse-media signal left as future scope
Liu et al., Isolation Forest (ICDM 2008) [8]	Anomaly detection by random partitioning	Linear time, no labels needed	No native explanation; contamination must be set	Implemented for licence-usage anomalies
Lundberg and Lee, SHAP (NeurIPS 2017) [9]	Unified feature attribution	Consistent local explanations; fast for trees	Cost grows with features; not causal	Implemented: reasons shown for each flagged licence
Wu et al., AutoGen (2023) [17]	Multi-agent LLM conversation framework	Modular and customisable	Conversation-driven; hard to audit	Contrast: event-driven agents with durable workflows

for review; this platform builds a confidence-gated pipeline around the extraction step. The legal NLP work stops at extracting clauses, whereas here the extracted terms drive a renewal workflow and sit beside a live vendor risk score that is shown before anyone signs. Isolation Forest and SHAP are general-purpose tools; the contribution here is their use for licence reclaim with explanations and a grace period. Finally, frameworks such as AutoGen favour flexibility and free-form dialogue, whereas procurement needs predictable, auditable and concurrency-safe behaviour, which is why the project uses an event bus and durable workflows for orchestration.

2.4 Commercial Procurement Platforms

Research prototypes are not the only point of comparison: procurement suites are a mature commercial market. A 2026 industry review [18] describes twelve products. Each was scored on the eight capability areas of this platform (document capture, vendor and fraud controls, invoice matching, approvals, inventory, software licences, contracts, supplier risk), counting partial coverage as half. The five with the largest overlap are Coupa (7 of 8) [19, 20], Zapro (6) [21], SAP Ariba (5.5) [22], Ivalua (4.5) [18] and TYASuite (4) [23]; the others cover at most three and a half areas. Table 2.4 compares them with this platform, using only what each vendor's public material states.

Table 2.4: Comparison with five commercial procurement platforms (✓ offered, ∼ partly, – not stated in the vendor's public material, October 2026)

Capability	**This platform**	**Coupa**	**Zapro**	**SAP Ariba**	**Ivalua**	**TYASuite**
AI document capture and extraction	✓upload, e-mail, folder	✓	✓	✓smart OCR	∼	✓
Classification with per-field confidence and review	✓	–	–	–	–	–
Measured error rate of automatic acceptance	✓(Section 5.4.6)	–	–	–	–	–
Invoice matching	✓line-level 3-way	✓	✓2/3-way	✓	✓	✓3-way
Duplicate and fraud checks	✓	∼	–	✓	–	✓
Bank-detail change needs a second person	✓	–	–	–	–	–
Requisitions and approvals	✓tiers, SoD, SLA	✓	✓	✓	✓	✓
Budget control	–	✓	✓	–	–	✓
Sourcing, RFQ and quote comparison	∼	✓	✓	✓	✓	✓
Catalogs and punch-out buying	–	✓	✓	✓	✓	–
Supplier portal or network	–	✓	∼	✓	∼	✓
Inventory	✓	✓	✓	–	∼	∼
Software-licence usage	✓ML anomaly	–	∼ tracking	–	–	–

Table 2.4 (continued)

Capability	**This platform**	**Coupa**	**Zapro**	**SAP Ariba**	**Ivalua**	**TYASuite**
Contract lifecycle	✓e-sign, renewals	✓	∼	✓	✓	–
ML clause or obligation extraction	✓Legal-BERT	✓	–	–	–	–
Supplier risk scoring	✓explainable	✓	∼	✓	∼	–
Sanctions screening	✓	✓	–	–	–	–
Indian tax compliance	∼ GSTIN, IFSC	–	–	–	–	✓GST, TDS
Payments	–	✓	∼	–	✓	✓
Spend analytics	∼	✓	✓	∼	–	✓
ERP integration	–	✓	✓	✓	✓	✓
Conversational AI or agents	∼ read-only	✓	✓	✓	–	–
Deployment	Self-hosted	SaaS	SaaS	SaaS	SaaS	SaaS

Coupa is the strongest competitor: it covers every area except software licences, and its AI is the most developed, with invoice capture, obligation extraction from signed contracts, and agents that monitor supplier risk and sanctions. Zapro is the closest peer in scale and is the only other product that tracks software licences, although without a usage model. The commercial suites offer several capabilities that this platform lacks: ERP integration, budget control before approval, supplier portals, sourcing events, catalog buying and payment execution, and, for TYASuite, Indian GST and TDS compliance. Conversely, none of the five describes a usage model for licence reclamation, dual control of bank-detail changes, or—most relevant here—an error rate for the documents they accept without review: vendors report extraction accuracy (96–99% in their own figures), not how often an automatically accepted document is wrong. These public descriptions are marketing material and were not verified independently.

2.5 Literature Summary and Research Gap

Table 2.5 summarises the techniques reviewed and where each is used in the proposed system.

Two gaps follow. The first is integration. Document extraction, clause analysis, anomaly detection and workflow orchestration are usually studied as separate problems and judged on benchmarks. Few systems join them into a single governed procurement pipeline, and fewer still do so with three properties in mind. The pipeline should be *fraud-aware*, with dual control of payment-detail changes and malware scanning that fails closed. It should be *accountable*, with an audit log of document, vendor, approval and contract actions, explanations for model outputs, and monitoring for model drift. And it should be *observable*, with per-service metrics, centralised logs that follow one correlation identifier across services, and a record of every event and notification.

The second gap concerns the decision at the centre of document automation: whether a document can skip review. Document-automation pipelines typically compare a confidence score with a fixed threshold, as this platform does, without knowing the error rate that the threshold admits. Distribution-free risk-control methods such as Learn-then-Test [24] choose such a threshold so that the error rate among accepted items stays below

Table 2.5: Techniques reviewed and their use in the proposed system

Area	**Best-known technique**	**Use in this system**
Layout and OCR	Docling, PaddleOCR, Tesseract	pdfplumber for text PDFs, Docling for scanned PDFs, PaddleOCR for images
Extraction cross-check	LayoutLMv3	Optional second reading; disagreement forces review
Classification	TF-IDF + logistic regression	Trained classifier with keyword fallback below 60% confidence
Clause extraction	Legal-BERT on CUAD	Primary extractor with regular-expression fallback
Licence analysis	Isolation Forest + SHAP	Anomaly score, explained reasons, reclaim after a grace period
Risk scoring	Classifier + rule overrides	Random Forest with GSTIN and sanctions overrides; PSI drift check
Orchestration	Temporal, locks, Kafka, outbox	Temporal workflows, Redis reservation lock, Redpanda with transactional outbox

a target with high probability, but they have been applied to document-internal confidence only. Whether evidence from the organisation's own records—the vendor master, open purchase orders and earlier invoices—lets more documents through at the same error rate has not been measured. This project builds the integrated pipeline and makes a first measurement of that question (Section 5.4.6).

Chapter 3

Proposed System

This chapter states the problem that motivated the project, defines what the work does and does not cover, and describes the proposed system: its architecture, agents, document pipeline, event flow, use cases and technology choices.

3.1 Problem Statement

In most organisations the IT procurement process is a chain of loosely connected manual steps. A purchase order or invoice arrives in whatever form the vendor chose, someone retypes its contents into a system, someone else checks the vendor, and an approval request travels by e-mail until somebody replies. Contracts are filed with their renewal dates noted on a calendar, and software licences are renewed without checking whether anyone still uses them. Each step is slow on its own, and together they leave room for costly mistakes and for fraud. Table 3.1 lists the main problems and their impact.

Table 3.1: Problems in the current procurement process and their impact

Problem	**Impact**
Documents arrive through disconnected channels in inconsistent formats	Manual data entry, delays and extraction errors
Vendor records are not de-duplicated and payment-detail changes are accepted on request	Duplicate vendors, BEC fraud and payments to the wrong account
Approvals are routed by hand with no SLA or stock checks	Stalled requests, overspending and two requests consuming the same stock
Software licence usage is not monitored, or is judged by a flat inactivity rule	Money spent on unused seats, or seats reclaimed by mistake
Contract terms are tracked informally	Silent auto-renewals and missed notice periods
Vendor risk is assessed once, at onboarding	Stale risk ratings
No single view of a request across tools	Difficult auditing and slow investigation of problems

These problems are connected. A vendor record that was never checked is exactly what makes a fraudulent bank-detail change possible, and an extraction error that goes unnoticed can become a duplicate payment, so fixing one step in isolation helps only a little. The problem addressed in this project is therefore: *to design and build an automated, event-driven procurement platform that reads unstructured procurement documents, links every document to a vendor record, routes approvals under spend and deadline controls, manages contracts and renewals, and keeps a vendor risk score up to date, while keeping automated decisions explainable and auditable and keeping a person in the loop for low-confidence results and sensitive changes.*

3.2 Scope

The project covers the path from an incoming procurement document or purchase request to a signed contract, the receipt of goods and the matching of the vendor's invoice. Within this scope the system scans and reads documents, matches vendors, routes approvals with spend-tier and SLA controls, reserves inventory, monitors licence usage, generates, signs and tracks contracts, scores vendor risk, and keeps users informed through e-mail notifications, all behind role-based access control and with metrics, logs and an audit trail.

The work does not include payment execution, ERP integration, adverse-media screening, live GSTIN registry lookups as a default (the client exists but is switched off), a third-party e-signature provider in the demonstrated configuration (Documenso and DocuSign clients exist but signing is done in the application), automatic retraining of models, or a production or Kubernetes deployment. All data is synthetic: the vendor risk training set, the licence usage logs, the classifier corpus and the sample documents are generated, and the CUAD contracts are used only to evaluate clause extraction.

3.3 Scope Design

The system design defines the structure, components, interfaces and information flow of the platform. Its scope covers the following areas:

- **System architecture:** the user interface, gateway, agent services, event bus, data stores and their relationships.
- **User interface design:** role-specific pages through which users submit requests, review extracted data, approve or reject requests, sign contracts and inspect vendors, licences and notifications.
- **Agent coordination:** how the agents cooperate through events, for example from document intake to purchase request, approval, contract and notification.
- **Data flow:** how information moves from user input through processing and analysis to stored records and messages.
- **External dependencies:** ClamAV signature updates, an SMTP service (Mailpit in development), the HaveIBeenPwned password range API, a public IFSC directory, the Frankfurter exchange-rate API, the OpenSanctions consolidated list, Hugging Face model downloads, and a local Ollama model for the assistant.
- **Output and decision support:** confidence scores, risk bands with contributing factors, anomaly reasons for licences, and the risk shown to a signer before signing.

The system assists procurement decisions and does not guarantee correct extraction, which is why low-confidence results are sent to human review.

3.4 Proposed System

3.4.1 Overview and Architecture

The proposed system is an event-driven platform of four cooperating agents. Each agent is one or more independent FastAPI services with their own data and responsibilities, and the agents communicate mainly through events on a Kafka-compatible bus (Redpanda) rather than by calling one another. Users reach the system through a React frontend and an Nginx gateway; every service verifies the caller's JWT and role itself. The services share one PostgreSQL database, in which each service owns and migrates its own tables, together with Redis for locks and caching and MinIO for document files. Figure 3.1 shows the overall structure.

This decomposition isolates concerns: a change to extraction does not touch the approval engine, and a slow parse cannot block an upload, because heavy work runs in background workers off the request path.

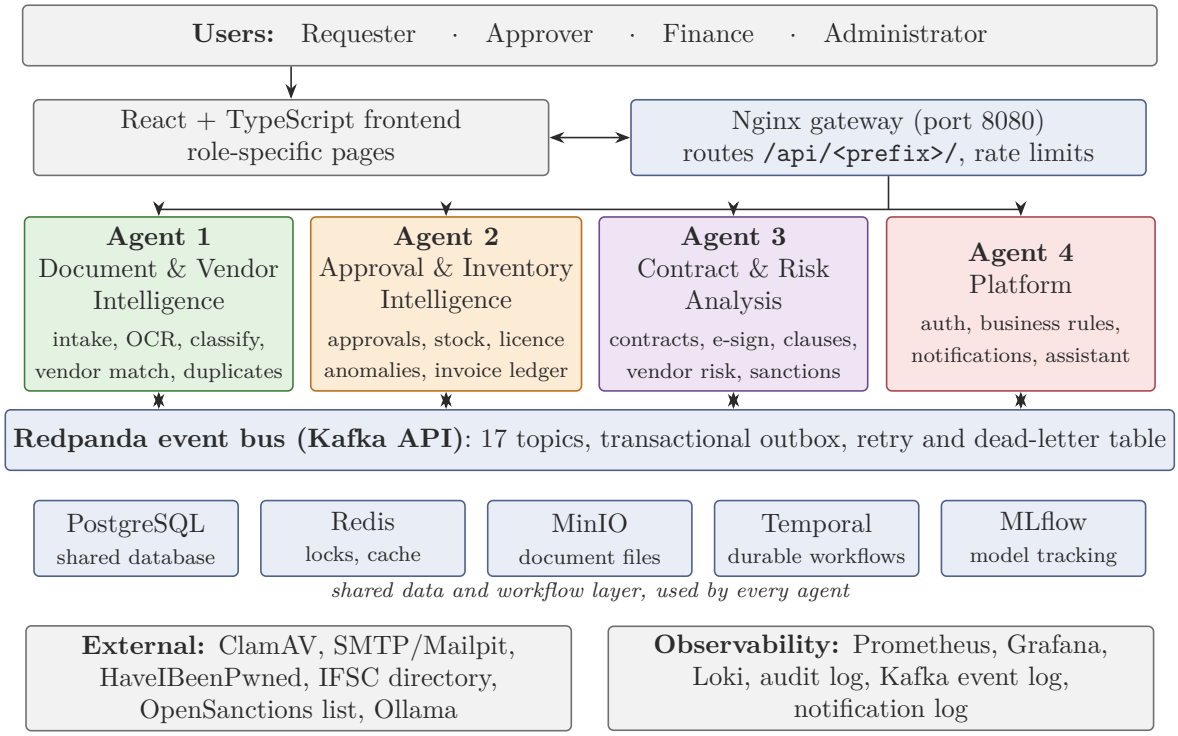


Figure 3.1: Overall architecture of the proposed system

3.4.2 The Four Agents

Table 3.2 summarises the agents, and the paragraphs that follow describe what each one does. In the sense defined in Section 2.2.8, each agent observes events, decides within the limits set by the business rules, acts, and defers to a person when its confidence is low or the action is sensitive.

Table 3.2: Summary of the four agents in the proposed system

Agent	Port	Main responsibility	Key techniques
Document and Vendor Intelligence	8001 (+ worker)	Document intake, reading, classification, field extraction, vendor matching and onboarding, duplicate detection, bank-detail control	ClamAV, pdfplumber, Docling, PaddleOCR, TF-IDF + logistic regression, LayoutLMv3, rapidfuzz
Approval and Inventory Intelligence	8002 (+ worker)	Approval routing and authority, SLA escalation, inventory reservation and reorder, licence anomalies and reclaim, invoice ledger	Temporal, Redis locks, Isolation Forest, SHAP
Contract and Risk Analysis	8003 (+ worker)	Contract generation and signing, clause extraction, renewal reminders, vendor risk scoring, sanctions screening, offboarding	Jinja2, Legal-BERT (CUAD), Random Forest, MLflow, PSI
Platform	8004, 8005, chatbot	Authentication and roles, business-rules engine, notifications, conversational assistant	JWT, argon2id, Jinja2, SMTP, Ollama (Llama 3.2 3B)

Document and Vendor Intelligence. This agent is the entry point for every document, whether uploaded, e-mailed to an intake address or dropped into a watched folder. It checks the file's size and real type, streams it through ClamAV and rejects it if the scanner flags it or cannot be reached, stores it in MinIO and publishes `document.ingested` in the same database transaction. A background worker reads, classifies and extracts the document, gives every field a confidence score and sends doubtful results to a review queue. The agent matches the vendor against existing records, books invoices against their purchase request, flags likely duplicates, and never applies new bank details directly: they wait as a pending change until a different user with the finance or admin role verifies them, and payment is held meanwhile.

Approval and Inventory Intelligence. This agent derives each request's approval chain from spend tiers held in the business-rules engine (automatic up to Rs. 500, a department manager up to Rs. 5,000, and a manager plus the finance head above that) and from an approval-authority matrix that also enforces separation of duties: nobody, including an administrator, can approve their own request. A Temporal workflow per request waits for the decision, reminds the approver at 50% and 80% of the SLA and escalates after 48 hours. Stock is reserved under a short Redis lock so that two concurrent requests cannot take the last unit, and a request for an item that is out of stock is created as back-ordered. An Isolation Forest scores licence usage; a licence above the anomaly threshold receives a reclaim request that waits through a seven-day grace period before it reaches an approver. When `contract.signed` arrives, the request moves to *ordered*; a user then marks the goods as received (*fulfilled*), and invoices are matched line by line against the order, with a 1% tolerance on the unit price and on the order's remaining balance before an invoice is held.

Contract and Risk Analysis. When a request is approved, this agent fills one of three Jinja2 templates (hardware purchase, SaaS subscription or professional services), renders the contract as text and PDF, and sends it for signature automatically. Signing happens in the application: an authorised approver, finance or admin user types or draws a signature, gives consent, and the contract and signer details are sealed with a SHA-256 hash recorded in the audit log and shown on a signature certificate. If the vendor is rated High or has not been scored, the signer must explicitly acknowledge the risk. An HMAC-verified, replay-protected webhook accepts signatures from external providers and is used by the tests. A Legal-BERT model reads the renewal term, notice period and expiration date from the contract text, and a Temporal workflow sends renewal reminders 60, 30 and 15 days before the notice deadline. The agent also scores each vendor's risk, screens vendor names against a sanctions list, runs a weekly PSI drift check, and handles vendor offboarding by flagging active contracts for review instead of closing them.

Platform. The fourth agent owns what no functional agent should. The authentication service issues 15-minute JWT access tokens and refresh tokens held in an httpOnly cookie (revoked on logout), hashes passwords with argon2id, verifies e-mail addresses, locks an account after repeated failures and rejects passwords found in known breaches. It also hosts the business-rules engine: 47 rules, such as spend tiers, thresholds and tolerances, that an administrator edits at run time and that services refresh when `business_rule.updated` is published. The notification service consumes 11 event topics, sends urgent messages immediately and batches routine ones into digests, retries failed deliveries and logs every message. Finally, a conversational assistant answers questions about requests, licences

and vendors using a local Llama 3.2 (3B) model through Ollama; it can only call six read-only tools, each executed with the asking user's own token, so it sees nothing the user could not see.

3.4.3 Document Processing Pipeline

The document pipeline turns an uploaded file into a structured, confidence-scored record (Figure 3.2). The upload returns as soon as the file is stored; a worker processes it in the background.

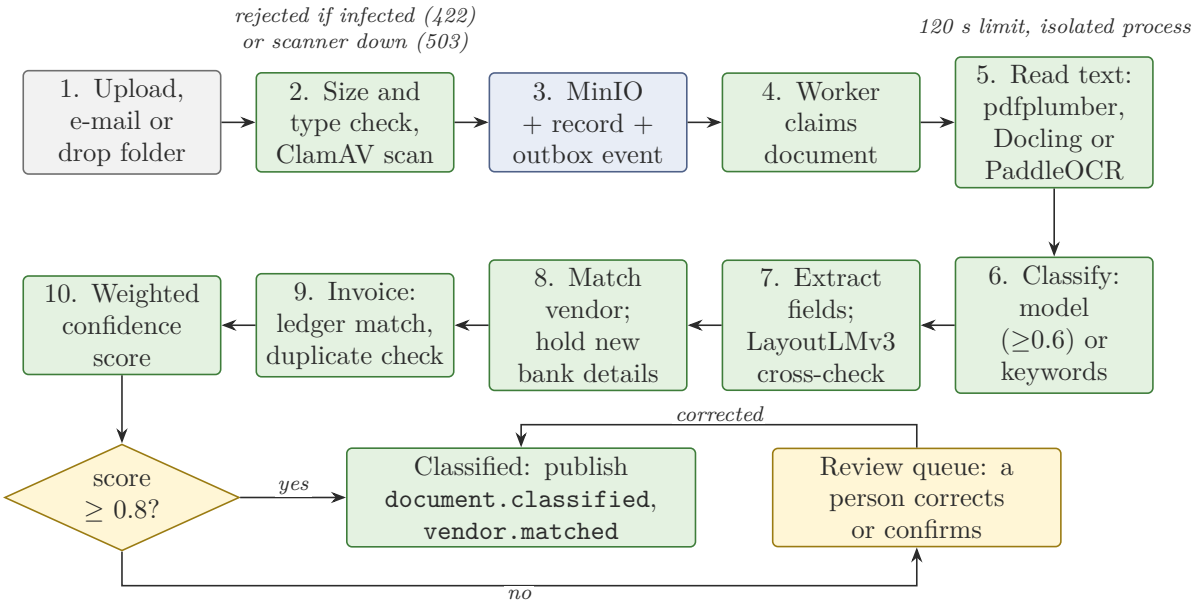


Figure 3.2: Document processing pipeline of the Document and Vendor Intelligence agent

A file is rejected before storage if it is empty, too large (25 MB at the gateway), of a type that its bytes do not confirm, infected, or if ClamAV cannot be reached, so the intake fails closed. After the file and its database row are stored, the worker reads it in a separate process with a 120-second limit: pdfplumber for PDFs with a usable text layer, Docling for scanned PDFs, and PaddleOCR for images after deskewing and upscaling by at most 1.5 times. The classifier labels the document as an invoice, purchase order, quotation or other paperwork, and regular expressions extract the vendor name, document number, date, currency, totals and line items. When LayoutLMv3 is installed it reads the document independently, and a disagreement forces review. The vendor name is normalised and matched against existing vendors with a fuzzy ratio of at least 88, or a new vendor record is created; first-seen bank details are held for verification. An invoice is booked against its purchase request and checked for duplicates: same vendor, an amount within 2%, a date within 7 days and a document number at least 85% similar (all three values are business rules). Finally, the per-field confidence scores are combined into an overall score (Section 5.2); a document scoring below 0.8 goes to the review queue, and reviewers' corrections are kept as per-vendor hints for later documents. Any dependency failure returns the document to the queue to be retried, up to five attempts.

3.4.4 Inter-Agent Communication and Workflow

Every event uses the same envelope (event ID, event type, timestamp, source service, correlation ID and payload), and the 17 topics and their payloads are fixed in a shared

contract document so that producers and consumers could be built in parallel. Each service authenticates to the broker with its own SASL/SCRAM account and may use only its own topics. Events are written to an outbox table in the same transaction as the change that caused them and relayed to the broker afterwards; consumers are idempotent, retry failures and move events that keep failing to a dead-letter table shown on the Control Panel. Figure 3.3 shows the main flows and Table 3.3 lists the topics.

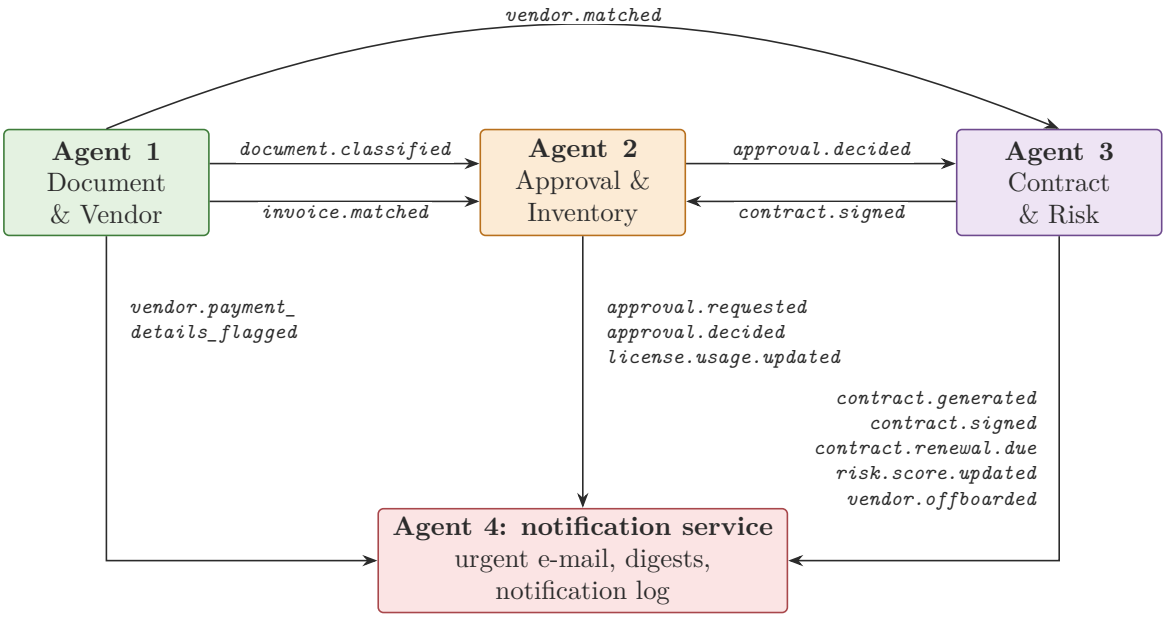


Figure 3.3: Main event flows between the agents. Rule changes (`business_rule.updated`) are broadcast to every agent and are omitted for clarity.

Table 3.3: Event topics exchanged between the agents

Topic	Published by	Consumed by and purpose
`document.ingested`	Agent 1	Agent 1 worker: starts the pipeline
`document.classified`	Agent 1	Agent 2: turns a quote into a purchase request; notifications
`vendor.matched`	Agent 1	Agent 3: scores the vendor's risk
`vendor.payment_details_flagged`	Agent 1	Notifications: urgent alert on a pending bank-detail change
`invoice.matched`	Agent 1	Agent 2: updates the invoice ledger and request status
`approval.requested`	Agent 2	Notifications: tells the approver
`approval.decided`	Agent 2	Agent 3: generates the contract; Agent 1: books invoices waiting for the request; notifications
`license.usage.updated`	Agent 2	Notifications: utilisation and anomaly alerts
`order.summary.generated`	Agent 2	Control Panel: hourly open-order summary
`contract.generated`	Agent 3	Notifications

Table 3.3 (continued)

Topic	Published by	Consumed by and purpose
`contract.signed`	Agent 3	Agent 2: request becomes *ordered*; notifications
`contract.renewal.due`	Agent 3	Notifications: 60, 30 and 15-day reminders (urgent at 15)
`risk.score.updated`	Agent 3	Notifications: urgent for the High band
`vendor.offboarded`	Agent 3	Notifications: offboarding summary and contracts flagged
`reminder.due`	Agents 1–3	Control Panel; e-mail delivery is future work
`business_rule.updated`	Agent 4	Agents 1–3: refresh their cached rules
`notification.send`	Any service	Notifications: generic message

In a typical run, a requester creates a purchase request, or a vendor quote is dropped into the intake folder and Agent 1 classifies it and Agent 2 creates the request from it. Agent 2 builds the approval chain and, once the last approver agrees, publishes `approval.decided`. Agent 3 generates the contract and sends it for signature, and its `contract.signed` event moves the request to *ordered*. The notification service follows all of these events, sending urgent alerts such as an approval request or a flagged bank-detail change immediately and batching routine ones into a digest. Every API request receives a correlation ID at the gateway, which is carried through the events and log lines of every service, so the whole journey of one request can be followed in Grafana.

3.4.5 Use Case View

Figure 3.4 presents the use cases of the four roles. The *Requester* raises purchase requests, uploads documents and follows their progress; the *Approver* decides the requests and licence reclaims assigned to them in the approval-authority matrix and can sign contracts; the *Finance* user verifies bank-detail changes, onboards vendors, maintains risk signals and can also approve and sign; and the *Administrator* manages users and business rules, monitors the system and offboards vendors. Administrators can perform every action, so the diagram shows each role only with the actions specific to it.

The use cases group into five phases:

1. **Authentication and business rules.** Users sign up, verify their e-mail and sign in; an administrator assigns roles and edits the business rules (spend tiers, SLA, thresholds and tolerances) on the Business Rules page, with every change kept in a history.

2. **Purchase and inventory.** A requester submits a purchase request, stock is checked and reserved, the approval workflow runs in Temporal, and the approver approves or rejects.

3. **Vendor intelligence.** Vendor documents are read and matched, new vendors are onboarded with identifier checks, bank details are verified by a second person, and a vendor risk score is calculated.

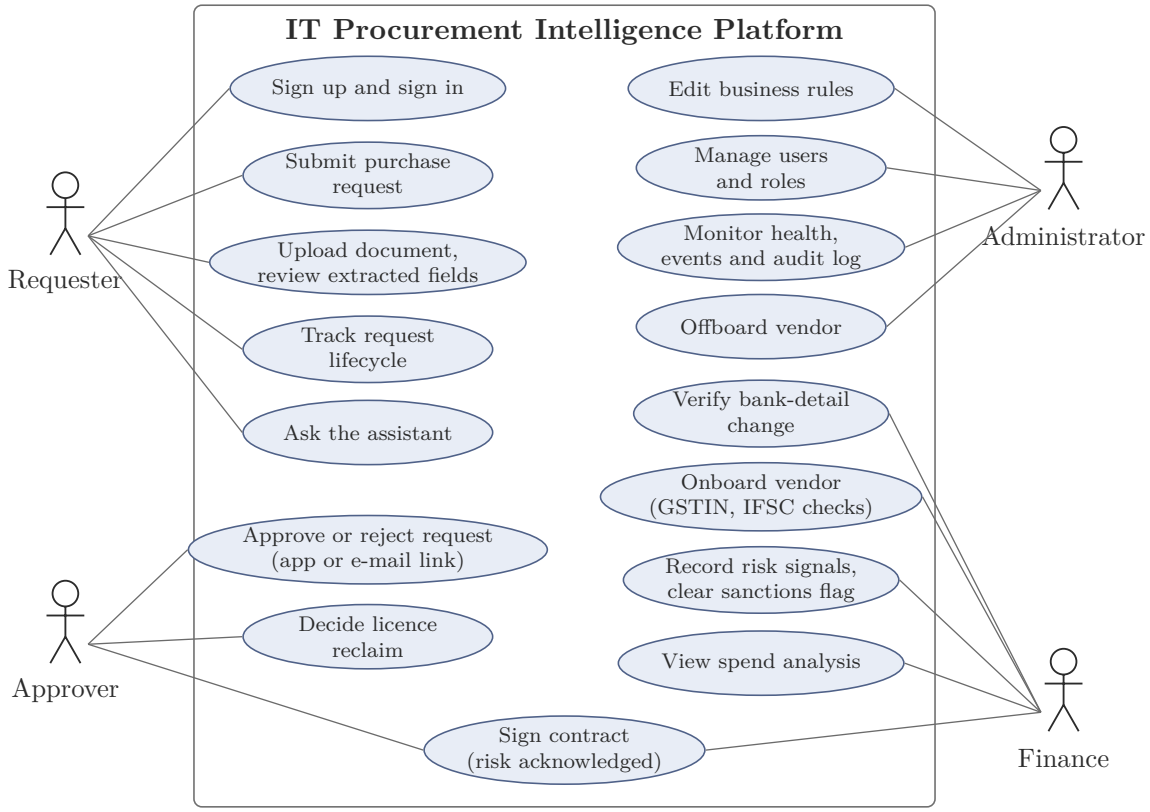


Figure 3.4: Use case diagram of the IT procurement platform

4. **Contract and risk.** A contract is generated for the approved request, sent for signature, signed in the application after the vendor's risk is shown, and renewal reminders are scheduled.

5. **Notification.** Approvals, flagged bank details, renewals, risk changes and signatures trigger e-mail notifications to the people concerned.

3.4.6 Key Design Principles

Four principles run through the design. The system is *fraud-aware*: uploads are scanned and rejected if the scanner is unavailable, bank details need a second person's verification, the webhook is accepted only with a valid HMAC signature and never twice, and signing a contract with a High-risk vendor requires an explicit acknowledgement. It is *human-centred*: results below the confidence threshold go to review, a licence reclaim waits for the holder and an approver, offboarding flags contracts rather than closing them, and the models never retrain themselves. It is *accountable*: significant actions are written to an audit log, identity is always taken from the JWT and never from a request body, business-rule changes are versioned, and every model-routing decision is logged. Finally, it is *observable*: every service exposes Prometheus metrics, logs carry a correlation ID, the Control Panel shows every Kafka event and every failed event, and a System Health page shows the state of the services.

3.4.7 Technology Stack

Table 3.4 lists the main technologies. Pretrained open-source components are used wherever possible; the models trained within the project are the document classifier, the

licence-anomaly model and the vendor risk model.

Table 3.4: Technology stack of the proposed system

Layer	**Technology**	**Purpose**
Frontend	React, TypeScript, Vite, Tailwind CSS	Role-based pages, one home page per item
Gateway	Nginx	Single entry point; routes `/api/<prefix>/`, rate limits, security headers
Backend services	FastAPI (Python), SQLAlchemy, Alembic, Docker Compose	One containerised service per agent, plus workers
Messaging	Redpanda (Kafka API), aiokafka, SASL/SCRAM	Events between agents, per-service topic permissions
Workflow engine	Temporal	Approval SLAs, reclaim grace period, renewal reminders
Data stores	PostgreSQL, Redis, MinIO	Records, locks and caches, document files
Document AI	pdfplumber, Docling, PaddleOCR, LayoutLMv3, rapidfuzz	Text extraction, OCR, cross-check, vendor and duplicate matching
Models	scikit-learn (logistic regression, Isolation Forest, Random Forest) [25], SHAP, Legal-BERT (Hugging Face Transformers), MLflow	Classification, licence anomalies, vendor risk, clause extraction, experiment tracking
Assistant	Ollama with Llama 3.2 (3B), tool calling	Read-only questions about requests, licences and vendors
Security	JWT, argon2id [26], HMAC-SHA256, ClamAV, HaveIBeenPwned	Authentication, password hashing, webhook verification, malware scanning, breached-password check
Monitoring	Prometheus, Grafana, Loki, Promtail, kafka-exporter	Metrics, dashboards, centralised logs by correlation ID, consumer lag
Notifications	Jinja2 templates, SMTP (Mailpit in development)	Templated e-mail

Chapter 4

Design of the System

This chapter describes the requirements, development approach, system models, resource requirements and architecture of the platform. It presents the functional and non-functional requirements, the data flows, the main UML views, the database design and the system layers.

4.1 Requirement Engineering

Requirement engineering identifies the capabilities expected from the platform and keeps them consistent during development. It is central here because the system is divided into services that must interoperate through shared event formats, database tables, REST conventions and authentication rules.

4.1.1 Requirement Elicitation

Requirements were identified from the following sources:

- **Existing practices and literature:** the review in Chapter 2 and the problem analysis in Chapter 3 identified the recurring problems: repeated manual data entry, unverified vendor details, duplicate invoices, delayed approvals, under-used licences, missed renewals and infrequent vendor-risk reviews.
- **Stakeholder analysis:** the users are requesters, approvers, finance users and system administrators, implemented as the four roles *requester*, *approver*, *finance* and *admin*.
- **Guide and mentor reviews:** reviews with the guides prioritised features and separated what could be built from what depends on paid accounts; the changes requested in a mentor review on 29 August 2026 were tracked to completion.
- **Review of tools and datasets:** document-processing tools, workflow engines and public datasets were tried against their expected inputs and practical limits. For example, PaddleOCR's native engine crashed the worker process on some platform and version combinations, which led to running it in an isolated subprocess rather than dropping it; and the CUAD dataset was chosen to evaluate clause extraction.

The requirements were then grouped into functional and non-functional requirements.

4.1.2 Software Lifecycle Model

The project followed an iterative and incremental model (Figure 4.1). This suited a system whose thresholds, approval rules, models and integration details had to be tested and refined during implementation. Each team member owned one service. The main practices were:

- **Contract-first design:** the Kafka event envelope and topic payloads, the base database schema, the REST response envelope and the authentication middleware were agreed before the services were built in parallel.
- **Service-wise increments:** the work was divided across the document and vendor, approval and inventory, contract and risk, and notification and authentication services.

- **Integration iterations:** the services were connected behind one gateway. Integration exposed problems in gateway routing, broker addressing, UUID column types and colliding migration tables, which were fixed at the shared level.
- **Continuous testing and audits:** unit and end-to-end tests were written alongside the features, and two quality audits re-tested the running system, with a regression test added for each defect fixed.
- **Repeatable deployment:** a single `run.sh` script builds and starts the whole containerised stack from a clean clone.

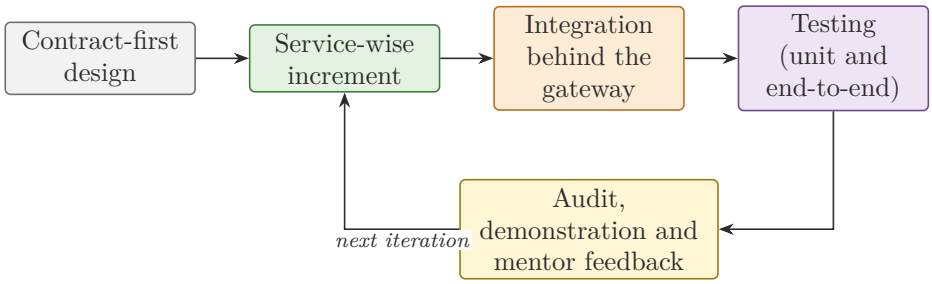


Figure 4.1: Iterative and incremental development cycle

4.1.3 Requirement Analysis

The requirements are grouped into functional requirements, which describe what the system does, and non-functional requirements, which describe the qualities and constraints it must satisfy. Every threshold mentioned below is a business rule that an administrator can change at run time; the values given are the current settings.

4.1.3.1 Functional Requirements

Table 4.1 lists the functional requirements and the agent responsible for each.

Table 4.1: Functional requirements

ID	Requirement	Agent
FR1	Users can sign up, confirm their e-mail address, sign in and receive a 15-minute access token and a refresh token that logout revokes. Every endpoint checks the token and the role, except the health and metrics endpoints and the HMAC-authenticated signature webhooks.	Platform
FR2	An administrator can view and edit the business rules (spend tiers, SLA, thresholds and tolerances); each change is versioned and reaches every service without a restart.	Platform
FR3	Documents can be uploaded, e-mailed to an intake address or dropped into a watched folder. Each file is size- and type-checked and scanned for malware before storage; it is rejected if the scanner is unavailable.	Document and Vendor
FR4	Documents are classified as invoice, purchase order, quotation or other; vendor, document number, date, currency, totals and line items are extracted.	Document and Vendor

Table 4.1 (continued)

ID	Requirement	Agent
FR5	Each extracted field receives a confidence score. Documents below the threshold (0.8) go to a review queue, where fields can be corrected or confirmed.	Document and Vendor
FR6	Vendors are matched by normalised name (fuzzy threshold 88); onboarding validates GSTIN format and check digit and IFSC codes and flags possible impostors.	Document and Vendor
FR7	New or changed bank details stay pending, with payment held, until verified by a different user with the finance or admin role.	Document and Vendor
FR8	Likely duplicate invoices are flagged (same vendor, amount within 2%, date within 7 days, document number at least 85% similar).	Document and Vendor
FR9	Purchase requests follow approval chains by spend tier (automatic up to Rs. 500, manager up to Rs. 5,000, manager and finance above) and an approval-authority matrix; nobody can approve their own request.	Approval and Inventory
FR10	Each request's approval runs as a durable Temporal workflow with reminders at 50% and 80% of the SLA; after 48 hours the request escalates to the next approver, and a timeout at the last level rejects it. Approvers can also decide from a single-use e-mail link for amounts up to Rs. 50,000.	Approval and Inventory
FR11	Stock is reserved under a Redis lock so that concurrent requests cannot reserve the same unit; a request for out-of-stock items is back-ordered, and reorder suggestions are computed.	Approval and Inventory
FR12	Licence usage is scored by an anomaly-detection model with per-licence explanations; an anomalous licence receives a reclaim request that waits through a seven-day grace period before approval.	Approval and Inventory
FR13	Invoices are matched line by line against the purchase request, and an invoice exceeding the price or remaining balance by more than 1% is held.	Approval and Inventory
FR14	Contracts are generated from templates after approval and sent for signature automatically; renewal type, notice period and end date are extracted, and reminders are scheduled 60, 30 and 15 days before the notice deadline.	Contract and Risk
FR15	Contracts are signed in the application with consent and a sealed audit record; signing for a High-risk or unscored vendor requires an acknowledgement. External signature callbacks are accepted only with a valid HMAC signature and only once.	Contract and Risk
FR16	Vendor risk is a score and a Low, Medium or High band with its contributing factors; a cancelled GSTIN or a potential sanctions match forces High until finance clears it with a reason.	Contract and Risk
FR17	The risk model is checked weekly for drift with the Population Stability Index; it is never retrained automatically.	Contract and Risk
FR18	Vendor offboarding revokes the vendor's access, flags active contracts for review and sets a data-retention flag instead of deleting records.	Contract and Risk
FR19	The notification service consumes events, sends templated e-mail, sends urgent items immediately and batches routine ones into digests, retries failures and keeps a searchable log.	Platform
FR20	An assistant answers questions about requests, licences and vendors using only read-only tools and the asking user's permissions.	Platform

Table 4.1 (continued)

ID	Requirement	Agent
FR21	The frontend provides role-specific navigation and dashboards, a request wizard, an approval inbox, one home page per request, document, vendor, licence and contract, and pages for notifications, spend analysis, business rules, users, audit activity, system health and the Kafka event log.	Frontend
FR22	Significant actions on documents, vendors, requests and contracts are written to an audit log that administrators can query.	All

4.1.3.2 Non-Functional Requirements

Table 4.2 lists the qualities and constraints the system must satisfy.

Table 4.2: Non-functional requirements

Category	Requirement
Security	JWT authentication and role checks in every service; identity taken only from the token; argon2id password hashing and breached-password check; account lockout and gateway rate limits; fail-closed malware scanning; HMAC-verified webhooks; dual control of payment details; per-service Kafka credentials and topic permissions; only the gateway port exposed.
Reliability	Approval, reclaim and renewal workflows are durable (Temporal). Events are published through a transactional outbox; consumers are idempotent, retry and dead-letter failures; a sweep re-queues stranded documents; a timed-out last approval is rejected, never approved by default.
Consistency	A common REST envelope and error format; a purchase-request state machine enforced in code and by a database constraint; idempotent migrations owned by each service.
Performance	Uploads return once the file is stored; reading, OCR and model inference run in background workers with time limits.
Traceability	Every event carries an event ID, source and correlation ID; logs from every service are collected centrally and searchable by correlation ID; significant actions are audited; model-routing decisions are logged.
Explainability	Confidence scores per field, risk factors per vendor, SHAP reasons per licence and the risk shown before signing.
Maintainability	Responsibilities separated by service; shared contracts documented; thresholds held as business rules, each with a behaviour test.
Portability	The stack is containerised and runs on macOS, Linux and Windows through WSL2.

Constraints and assumptions. Payment execution, ERP integration, adverse-media screening, a third-party e-signature provider and automatic model retraining are outside the scope. Kafka traffic is authenticated but not encrypted, which is acceptable only on a single host. The vendor risk data, licence usage logs, classifier corpus and sample documents are synthetic.

4.1.4 UML Diagrams and Data Flow Diagrams

4.1.4.1 Use Case Diagram

The use case diagram is shown in Figure 3.4 in Chapter 3. It identifies the Requester, Approver, Finance user and Administrator and groups their activities into authentication

and rules, purchase and inventory, vendor intelligence, contract and risk, and notification.

4.1.4.2 Data Flow Diagram – Level 0

The context diagram (Figure 4.2) shows the platform as a single process and its exchanges with the external entities: the three groups of users, vendors who send documents, the e-mail service, and the external lookup services.

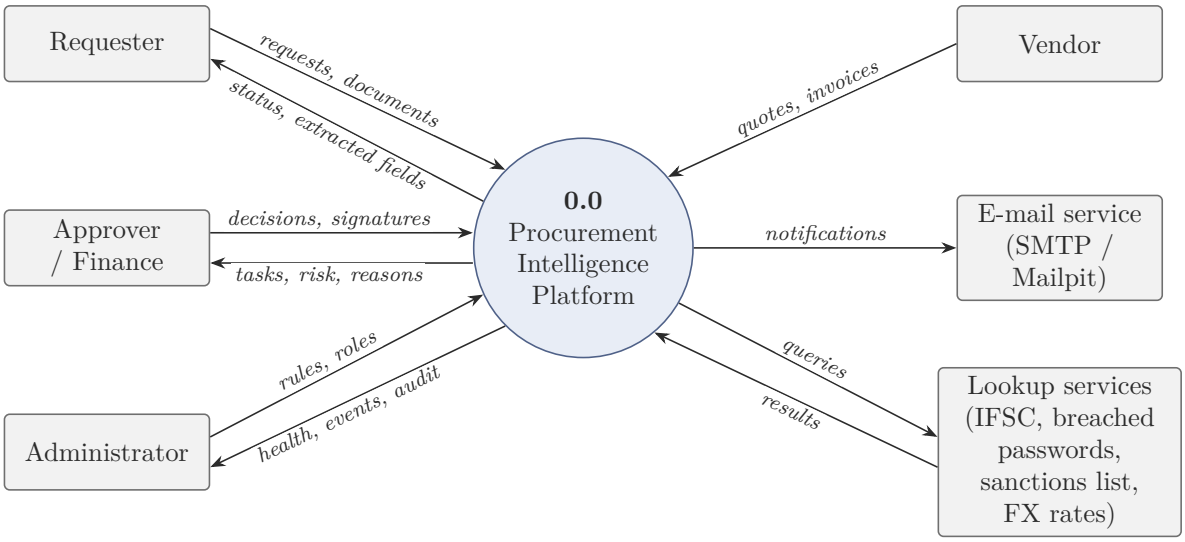


Figure 4.2: Level 0 data flow diagram (context diagram)

4.1.4.3 Data Flow Diagram – Level 1

The Level 1 diagram (Figure 4.3) decomposes the platform into the processes of Table 4.3. Processes 2.0 to 5.0 exchange events through the messaging layer, process 6.0 consumes them, and process 1.0 issues the tokens and rules that the others check.

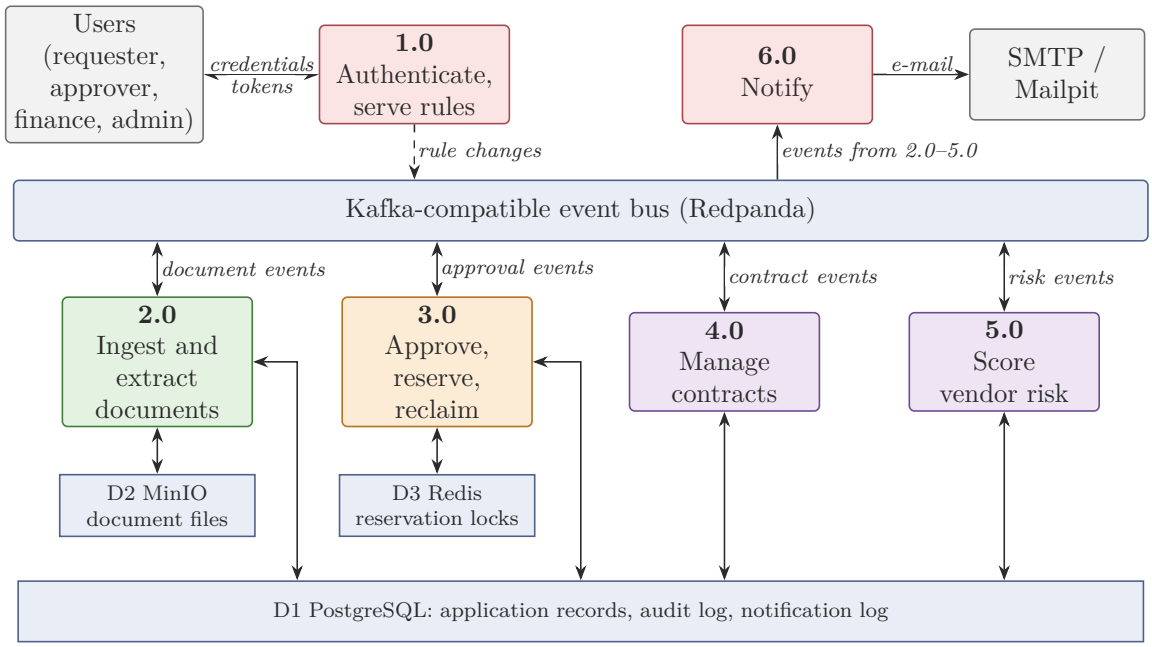


Figure 4.3: Level 1 data flow diagram

Table 4.3: Level 1 DFD processes

Process	Name	Inputs	Outputs
1.0	Authenticate, serve rules	Credentials, refresh token, rule edits	Access and refresh tokens, role, rule changes
2.0	Ingest and extract documents	Uploaded, e-mailed or dropped file	Classified document, vendor match, review-queue items, invoice match
3.0	Approve, reserve, reclaim	Purchase request, decisions, usage logs	Decisions, reservations, escalations, reclaim requests
4.0	Manage contracts	Approved request, signature	Contract text and PDF, signature record, renewal reminders
5.0	Score vendor risk	Vendor signals, sanctions list	Risk score, band, factors, overrides, drift flag
6.0	Notify	Events from 2.0–5.0	E-mail, digests, notification log

4.1.4.4 Sequence Diagram

Figure 4.4 follows one purchase request from creation to signature. It shows the synchronous calls through the gateway, the asynchronous events, and the point at which the Temporal workflow hands the decision back to the service.

4.1.4.5 Activity Diagram: Document Processing

The activity diagram (Figure 4.5) shows the decisions taken for one document. The upload is rejected if the file fails a check, is infected or cannot be scanned; a file that cannot be read fails with a message for the uploader. A readable file is classified, extracted, matched and scored. A document meeting the threshold, with no rule forcing review, is completed; otherwise it waits in the review queue until a reviewer corrects or confirms it.

4.1.4.6 Purchase-Request State Machine

A purchase request moves through the states of Figure 4.6. The allowed transitions are defined once in a shared module and checked on every status change, and the database rejects any status outside the list. An event that arrives after the request has already moved past the state it would cause (for example a late `contract.signed`) is recognised and ignored rather than treated as an error.

4.1.4.7 Entity Relationship Diagram

All services share one PostgreSQL database of about fifty tables; each service creates and migrates its own. Figure 4.7 shows the main entities and their foreign keys as they exist in the running database. The vendor is the central entity: documents, purchase requests, contracts, licences, risk scores, sanctions screenings and payment-change requests all refer to it. The `inventory` table has no foreign key: requests refer to stock by SKU inside their items. Supporting tables, not shown, hold users and tokens, business rules and their history, the event outbox, inbox and dead-letter table, the Kafka event log, notification and model-routing logs, and the reviewer-feedback tables.

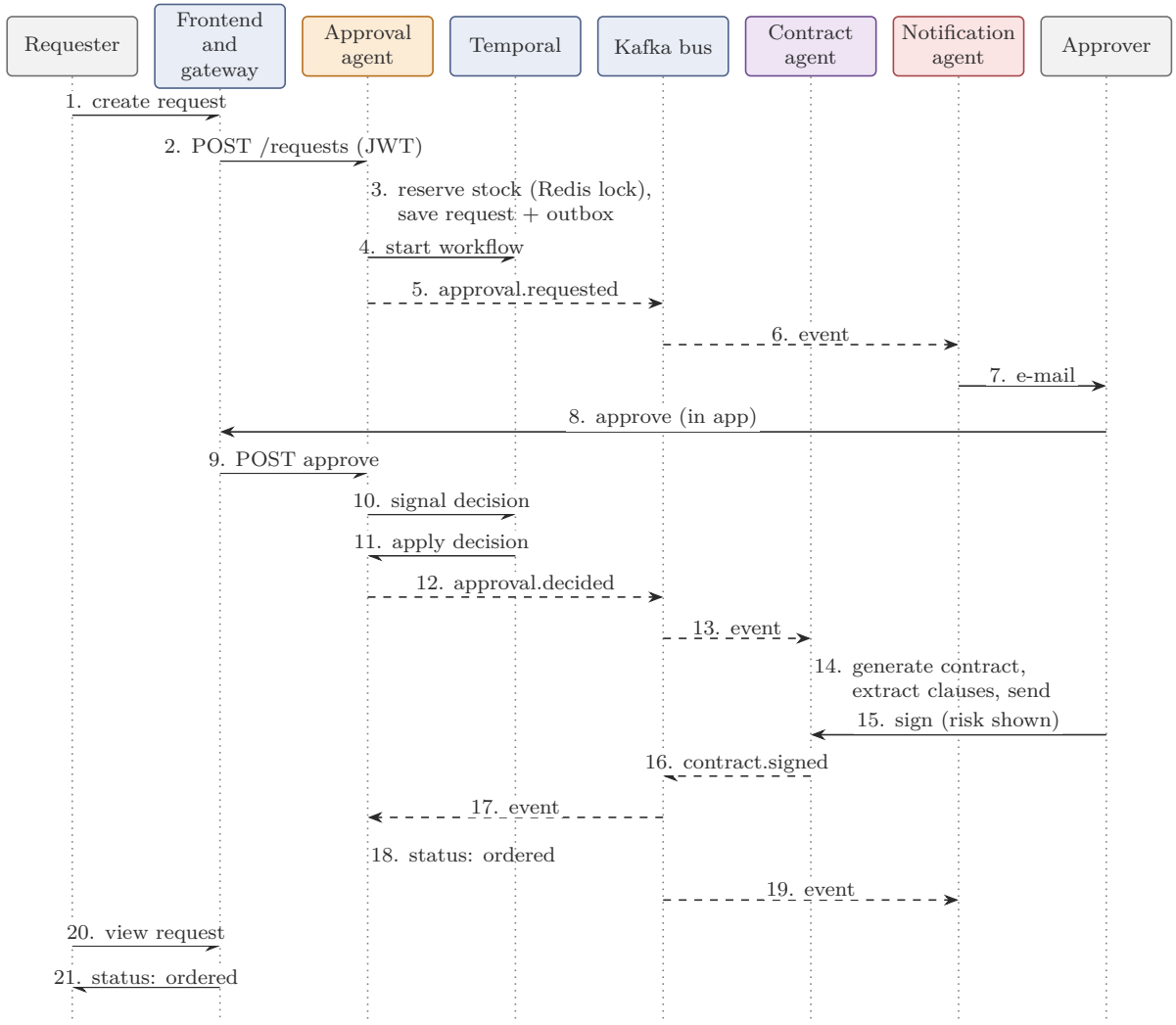


Figure 4.4: Sequence diagram of the purchase-request journey (solid: synchronous call; dashed: asynchronous event)

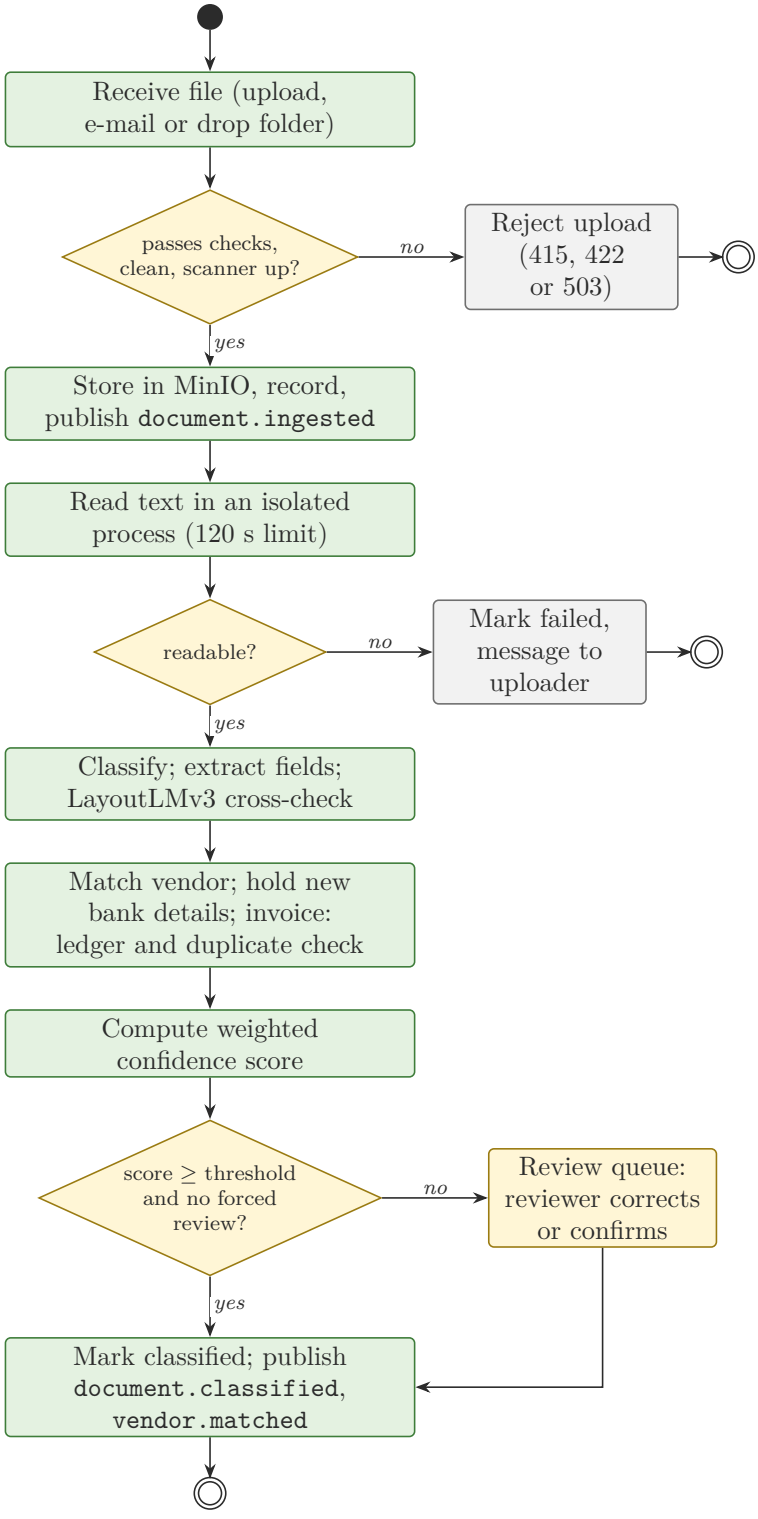


Figure 4.5: Activity diagram of document processing

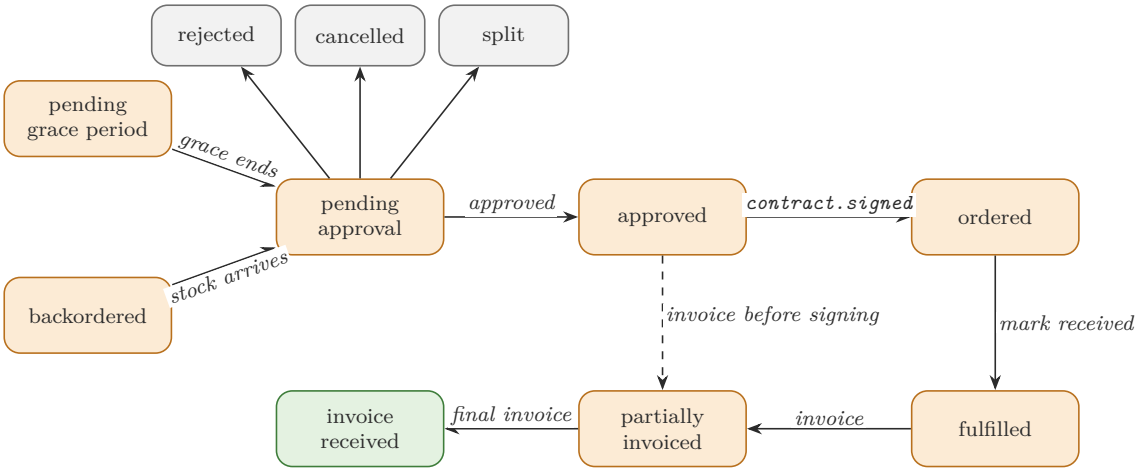


Figure 4.6: State machine of a purchase request (main transitions; stored state names use underscores, for example `pending_approval`; a request can also be cancelled from the waiting states)

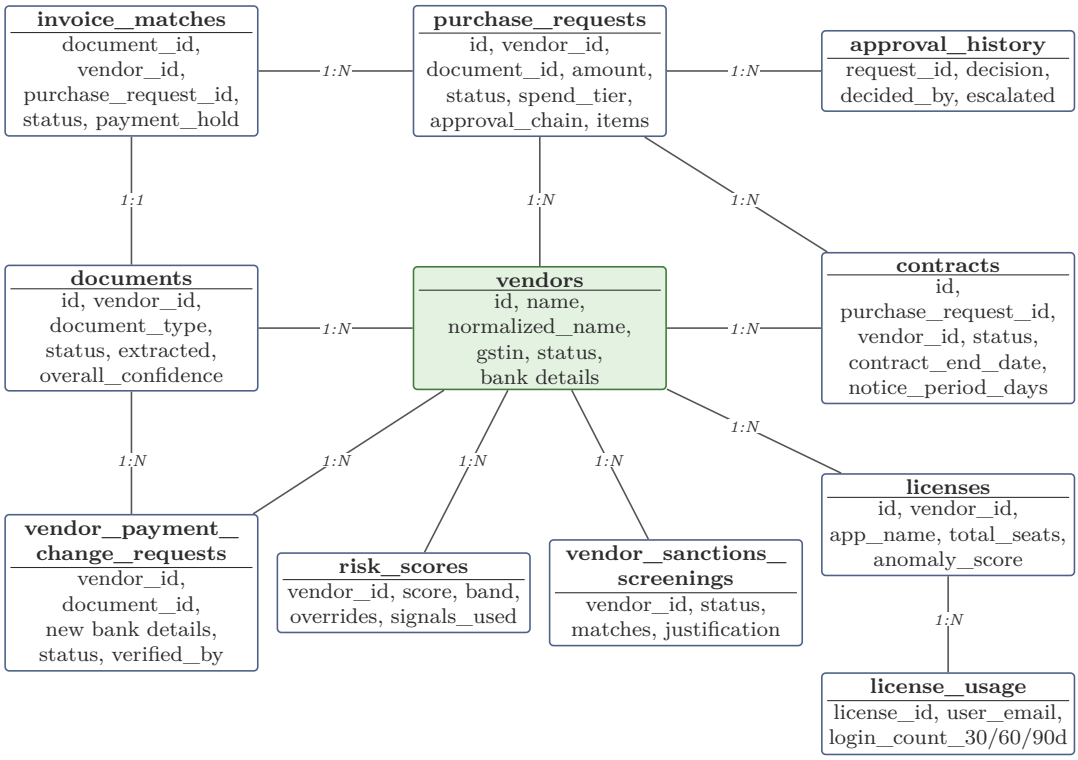


Figure 4.7: Entity relationship diagram of the main entities (foreign keys as in the running database; `invoice_matches` also refers to `vendors`)

4.1.5 Cost Analysis

The project uses only open-source software and free API tiers, so it incurs no software-licence cost, and it was developed on the team's own laptops. The potential costs are team effort, paid API tiers if live registry lookups are enabled, cloud hosting, and a production e-mail service. The estimates in Table 4.4 are indicative and should be revised against real quotations.

Table 4.4: Indicative cost analysis

Item	Basis	Indicative cost
Software licences	Python, FastAPI, PostgreSQL, Redis, Redpanda, MinIO, Temporal, MLflow, Prometheus, Grafana, Loki, ClamAV, Docling, PaddleOCR, Hugging Face models, Ollama, React and Docker are open source	Rs. 0
Development hardware	Existing student laptops	Rs. 0 additional
External APIs	HaveIBeenPwned range API, IFSC directory and Frankfurter exchange rates are free; the OpenSanctions list is free for non-commercial use only (CC BY-NC 4.0) and needs a licence for commercial use; live GSTIN lookups depend on the provider's quota	Rs. 0 in the project; depends on usage and licensing in production
Team effort	Part-time project work by four students	Not monetised
Optional cloud hosting	One virtual machine with about 4–8 vCPU and 16 GB RAM, plus storage and backups	Approx. Rs. 6,000–10,000 per month
E-mail delivery	Production SMTP service instead of the development mail catcher	A few hundred rupees per month at pilot volume

A meaningful break-even estimate needs measured document volumes, licence costs and the time saved or duplicate payments avoided. Those figures are not available, so no return-on-investment claim is made.

4.1.6 Hardware and Software Requirements

The stack runs 24 containers on one Docker host. Measured on 16 October 2026, it used about 3.2 GB of RAM when idle and more while reading scanned documents, and its images occupied about 15 GB before build cache and data volumes. ClamAV alone needs about 1 GB of memory. Table 4.5 gives the resulting requirements and Table 4.6 the software.

4.2 System Architecture

The platform follows a modular, event-driven architecture in which independent services each own one procurement function. The frontend talks to the services through the gateway with synchronous HTTP requests where an immediate answer is needed, while the services talk to one another mainly through events. Temporal runs the long-lived workflows; PostgreSQL holds the records, Redis the locks and caches, and MinIO the files. Background workers consume events and run slow tasks, so users get a response without waiting for every downstream step.

Table 4.5: Hardware requirements

Component	**Minimum**	**Recommended**
Processor	4 cores	8 cores
Memory	8 GB available to Docker	16 GB RAM
Storage	40 GB free	80 GB free for images, models (about 1 GB), sanctions data and volumes
Network	Required for the first build, model and sanctions downloads, and sign-up's breached-password check	Broadband connection
GPU	Not required	Optional, for faster OCR and transformer inference

Table 4.6: Software requirements

Category	**Requirement**
Operating system	macOS, Ubuntu/Linux, or Windows 10/11 with WSL2
Container runtime	Docker Engine or Docker Desktop with Compose v2
Backend	Python 3.11, FastAPI, SQLAlchemy, Alembic, pytest, scikit-learn, SHAP, Transformers, rapidfuzz, Jinja2
Frontend	React, TypeScript, Vite and Tailwind CSS, built in a Node 20 container
Data and messaging	PostgreSQL, Redis, Redpanda (Kafka API) and MinIO
Workflow and ML	Temporal, MLflow
Document processing	pdfplumber, Docling, PaddleOCR, LayoutLMv3 (optional download), ClamAV
Monitoring and mail	Prometheus, Grafana, Loki, Promtail, kafka-exporter, Mailpit
Assistant	Ollama on the host with the Llama 3.2 (3B) model
Gateway and browser	Nginx; a current Chrome, Edge, Firefox or Safari

4.2.1 UI/UX Design

The frontend is a React and TypeScript single-page application built with Vite and Tailwind CSS. Nginx serves it and routes its API calls, so the page and the API share one origin. The interface follows these principles:

1. **Role-aware navigation:** each role sees its own navigation and dashboard. Frontend route checks are only for usability; every endpoint enforces authorisation itself.
2. **One home page per item:** a request, document, vendor, licence and contract each have one page where all their actions live; lists, dashboards and the approval inbox link to it. A request page shows a clickable lifecycle whose steps open their details and actions.
3. **Clear asynchronous status:** decisions and document processing complete in background workers, so the interface polls briefly after an action and shows each step as it settles; a step is shown as in progress only while an agent is actually working on it.
4. **Human review:** extracted fields are shown with their confidence and can be corrected or confirmed; risk factors, licence anomaly reasons and the vendor's risk at signing are always visible.
5. **Honest limits:** where a feature has no backend support the interface shows it as unavailable rather than pretending it succeeded.
6. **Demonstration support:** the sign-in page offers one-click demo accounts in development builds, and a command palette (Ctrl/Cmd+K) searches loaded requests, vendors, contracts and documents.

Screenshots of the implemented screens are given in Section 5.1, and Table 4.7 lists the principal screens.

4.2.2 Block Diagram

The block diagram (Figure 4.8) shows the deployed components and their connections on the single Docker host. Table 4.8 describes each layer.

4.2.3 Main Data Paths

The principal data paths are:

1. **User access:** the browser loads the frontend from Nginx; API calls are routed to the right service after the gateway strips the `/api/<prefix>/` segment.
2. **Document processing:** the Document and Vendor agent checks and scans an upload, stores it in MinIO and publishes `document.ingested` through the outbox. Its worker reads, classifies and extracts the document and publishes `document.classified` and `vendor.matched`.
3. **Approval processing:** the Approval and Inventory agent reserves stock, saves the request and starts a Temporal workflow. An approve or reject call signals the workflow; the worker applies the decision, updates the status and publishes `approval.decided`.

Table 4.7: Principal screens and their purposes

Screen	User role	Purpose
Sign in, sign up, password reset	All	Account access and e-mail verification
Dashboards	Each role	Own requests, items awaiting approval, spend overview or system overview
New request wizard	Requester	Enter details, choose a vendor, optionally upload a document with live pipeline status, review extracted fields and submit
Request page	All, by permission	Clickable lifecycle, approval, procurement checks, invoice ledger, contract and signature
Approval inbox	Approver, finance	Pending requests and reclaims for the levels assigned to the user, with SLA status
Document page	Requester, approver, finance, admin	Preview, extracted fields with confidence, correction and confirmation
Vendor page and onboarding	Finance, admin	Risk score and factors, risk signals, bank-detail verification, sanctions status, onboarding checks
Licence page	Approver, finance, admin	Usage trend, anomaly score and reasons, open reclaim request
Contract page	Approver, finance, admin	Contract text, renewal dates, signing dialog, signed copy and certificate
Inventory, spend analysis, risk	Finance, admin	Stock and reorder suggestions, spend by category, vendor risk bands and drift
Business rules, users	Admin	Edit rules with history; manage roles
Control Panel, Controls, System Health, Audit	Admin (Controls: finance too)	Kafka event log and failed events, platform controls, service and model-routing status, audit trail
Notifications, account	All	Notification history; password change and sign-out everywhere

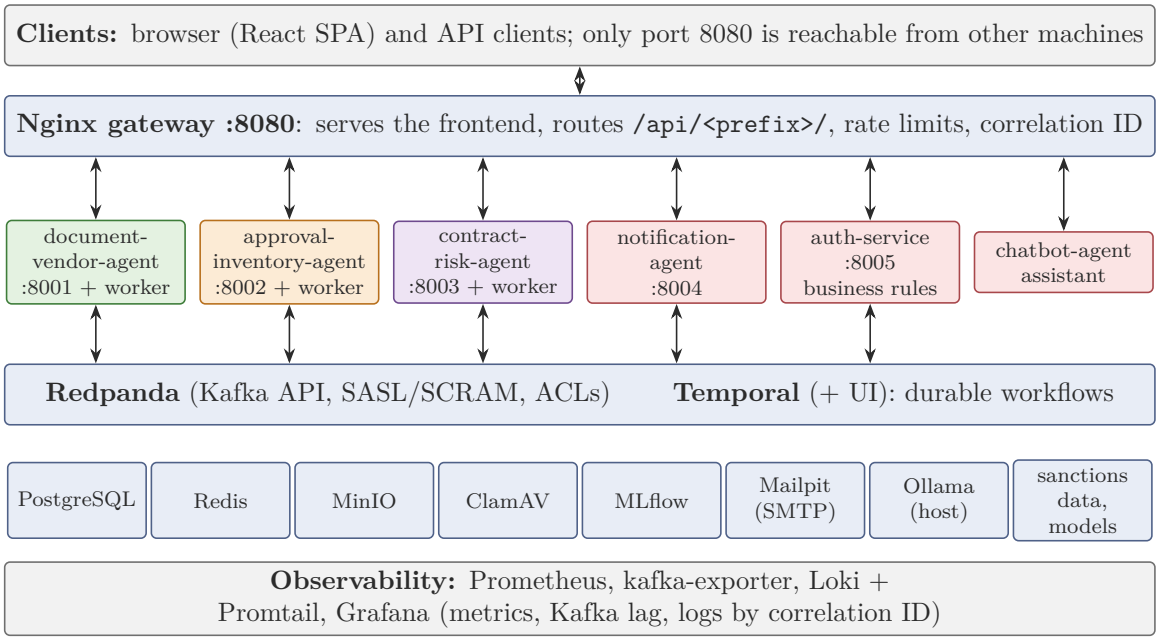


Figure 4.8: Block diagram of the deployed system

Table 4.8: Architectural layers

Layer	Components and responsibilities
Clients	The React application and API clients reach the platform only through the gateway on port 8080.
Gateway	Nginx serves the frontend, routes API paths to the services, limits request rates on authentication endpoints and assigns a correlation ID.
Application services	Document and Vendor agent (8001), Approval and Inventory agent (8002), Contract and Risk agent (8003), notification agent (8004), authentication and business-rules service (8005) and the assistant. Every service checks the JWT and role, except health, metrics and the HMAC-signed webhooks.
Background workers	The document, approval and contract services each run a worker that consumes events, runs Temporal workflows and performs slow tasks such as OCR and model inference.
Messaging and orchestration	Redpanda carries the events with per-service credentials; Temporal runs the approval, reclaim and renewal workflows.
Data and platform services	PostgreSQL holds the records, Redis the reservation locks and rule caches, and MinIO the files. ClamAV scans uploads, MLflow tracks the risk model and drift, Mailpit receives development e-mail, and Ollama runs the assistant's model on the host.
Observability	Prometheus collects metrics, Loki collects logs, and Grafana shows request rates, latency, errors, Kafka consumer lag and the logs of one correlation ID across services.

4. **Contract processing:** the Contract and Risk agent generates the contract for an approved request and sends it for signature. When it is signed, `contract.signed` moves the request to *ordered*; marking the goods as received makes it *fulfilled*, and the vendor's invoice is matched against it.

5. **Risk assessment and notifications:** the Contract and Risk agent publishes risk, renewal and offboarding events; the notification agent consumes them and sends e-mail.

6. **Supporting services:** PostgreSQL stores the records, Redis guards reservations, MinIO stores files, and MLflow records the risk model's training runs and drift checks.

Together, these paths show how synchronous API calls, asynchronous events and durable workflows combine to carry a request from intake to completion.

Chapter 5

Result and Discussion

This chapter presents the implemented system, its testing, and the evaluation of its models and of its document review gate. Screenshots were taken from the running system on 20 October 2026 with the synthetic demonstration data; test results are dated.

5.1 Screenshots of the System

Every item in the frontend has one page that holds all its actions; lists, dashboards and the approval inbox link to it. Figures 5.1 to 5.6 show a request (Figure 5.1), a document held for review (Figure 5.2), a vendor (Figure 5.3), a licence (Figure 5.4), a signed contract (Figure 5.5) and the Control Panel, which shows the agents' events as they happen (Figure 5.6).

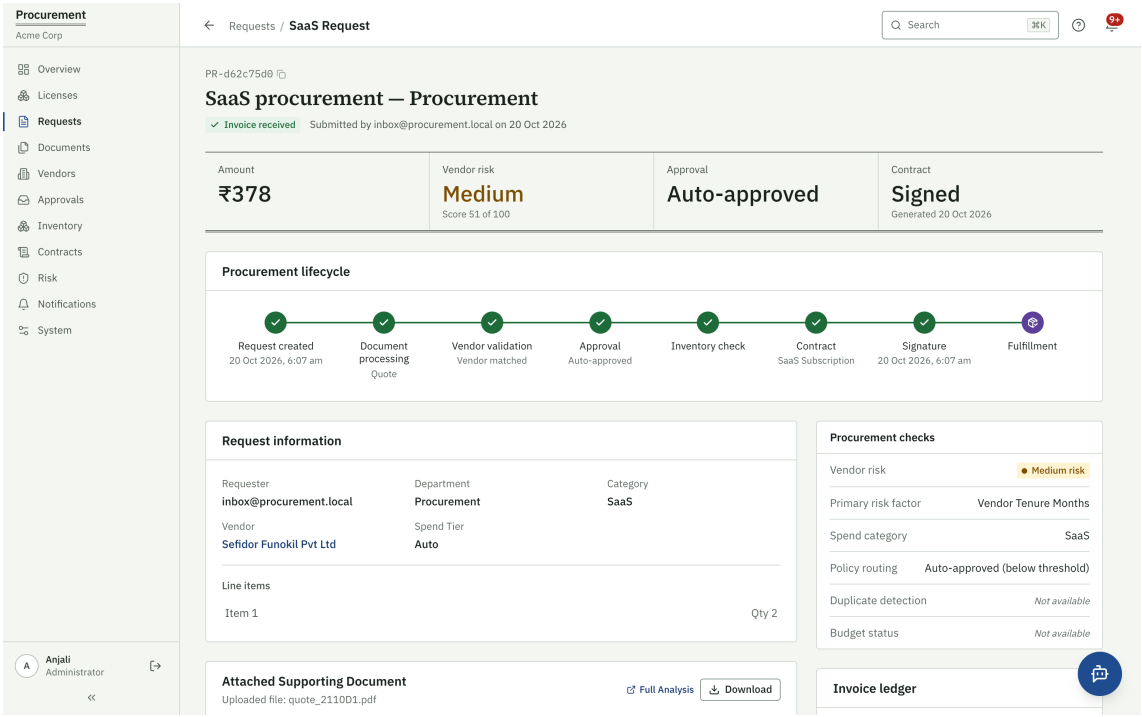


Figure 5.1: Request page of a SaaS purchase created from a dropped vendor quote: key figures, the completed lifecycle from request to invoice (each step opens its details), and the procurement checks

5.2 Sample Code (Main Logic)

Three excerpts of the implementation follow, unchanged except for removed log lines (marked "…").

Confidence gate. The worker discounts each field's extraction confidence c_f by the quality $q \in [0, 1]$ of the text it was read from, and combines the field scores with fixed weights w_f (total 0.30, vendor name 0.20, classification 0.15, document number 0.15, date 0.10, line items 0.10):

$$s_f = c_f \cdot q, \qquad S = \frac{\sum_{f \in F} w_f \, s_f}{\sum_{f \in F} w_f}, \qquad \text{review if } S < \theta, \tag{5.1}$$

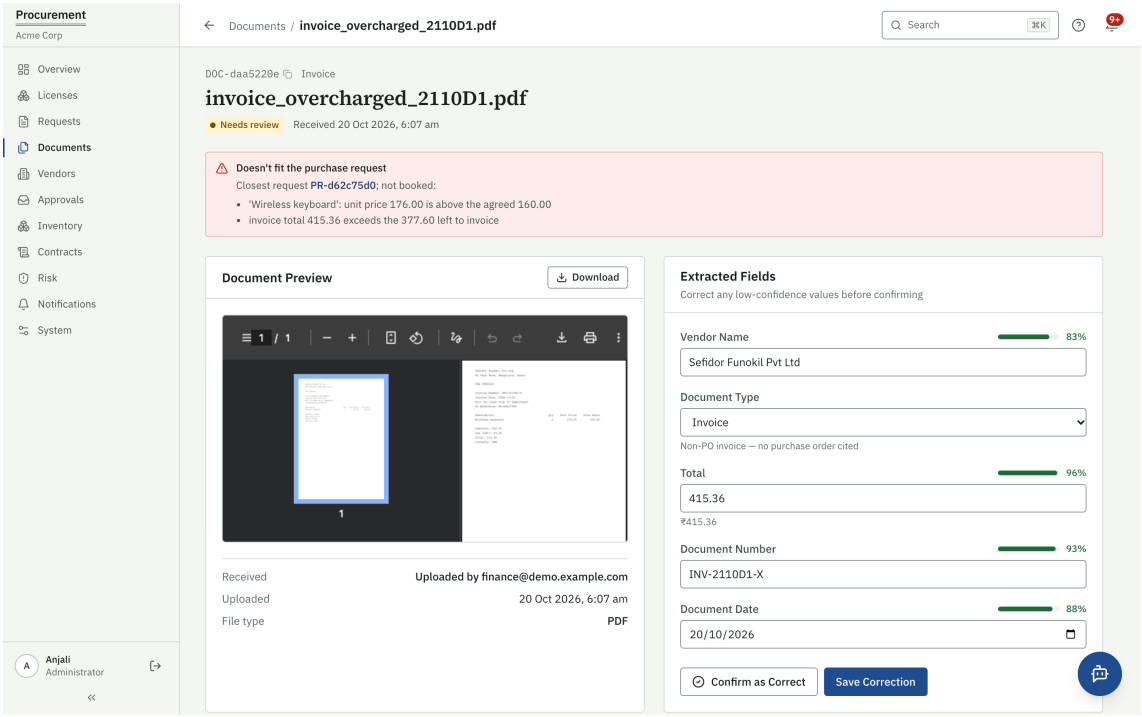


Figure 5.2: Document page of an invoice held for review: the three-way match found a unit price above the agreed one and a total above the balance left on the request; extracted fields are shown with their confidence for correction or confirmation

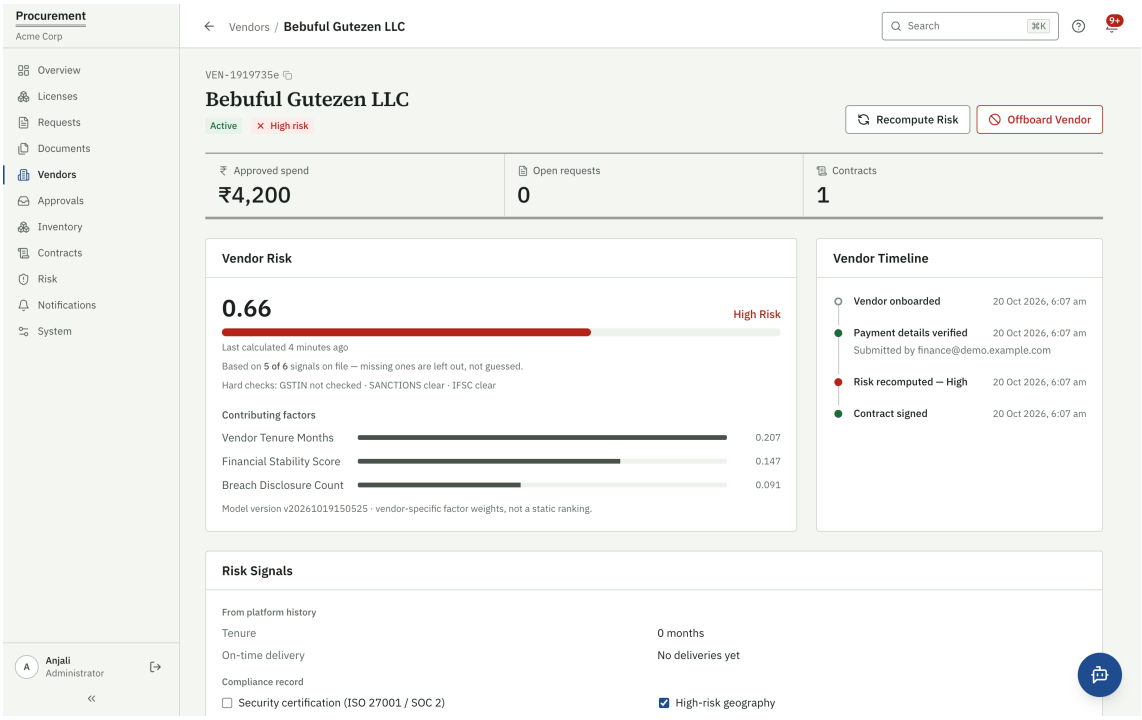


Figure 5.3: Vendor page: risk score (0.66, High) from five of six signals, the hard checks (GSTIN, sanctions, IFSC), the contributing factors and the vendor's timeline

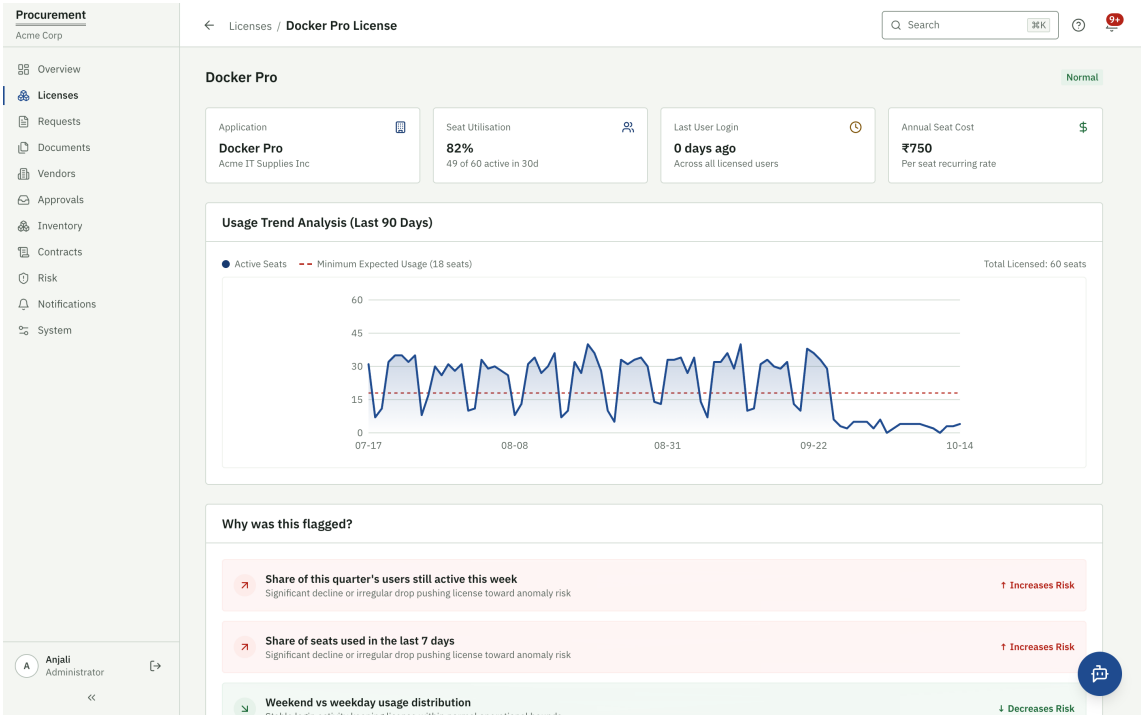


Figure 5.4: Licence page: seat utilisation, the 90-day usage trend with its late drop, and the SHAP explanation of the anomaly score. The score (0.32) is just below the watch threshold (0.34), so the licence is still rated Normal

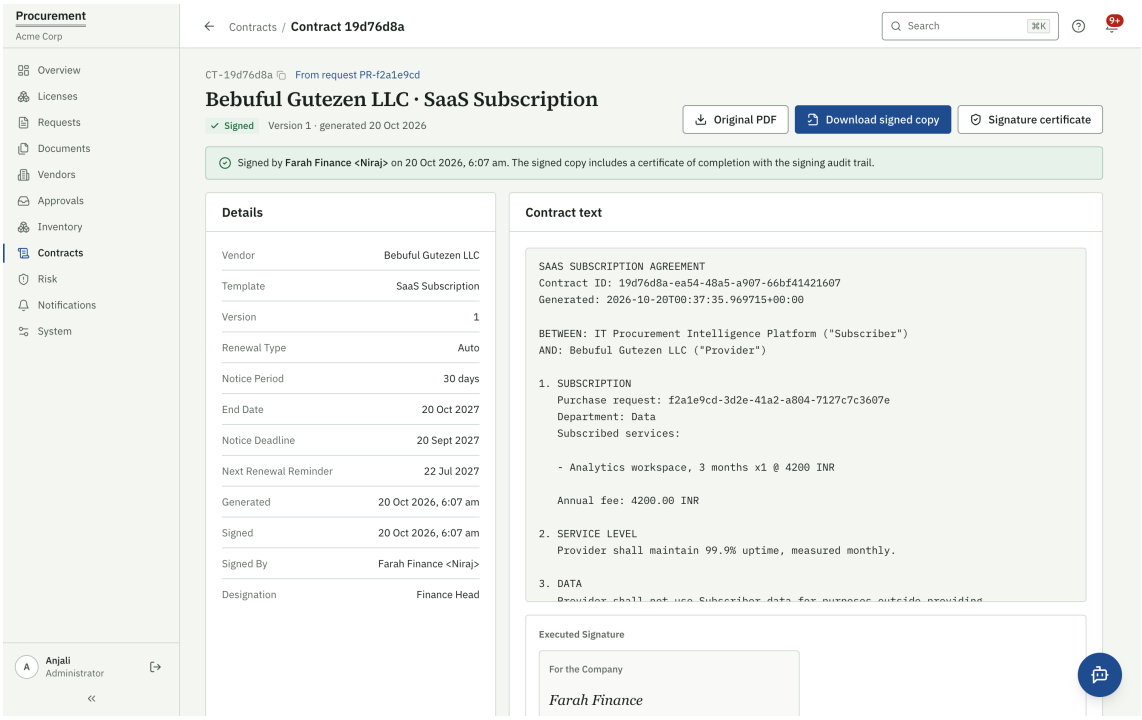


Figure 5.5: Contract page of a signed SaaS subscription: signer and signing record, renewal type, notice period and deadline, the next renewal reminder, and the generated contract text

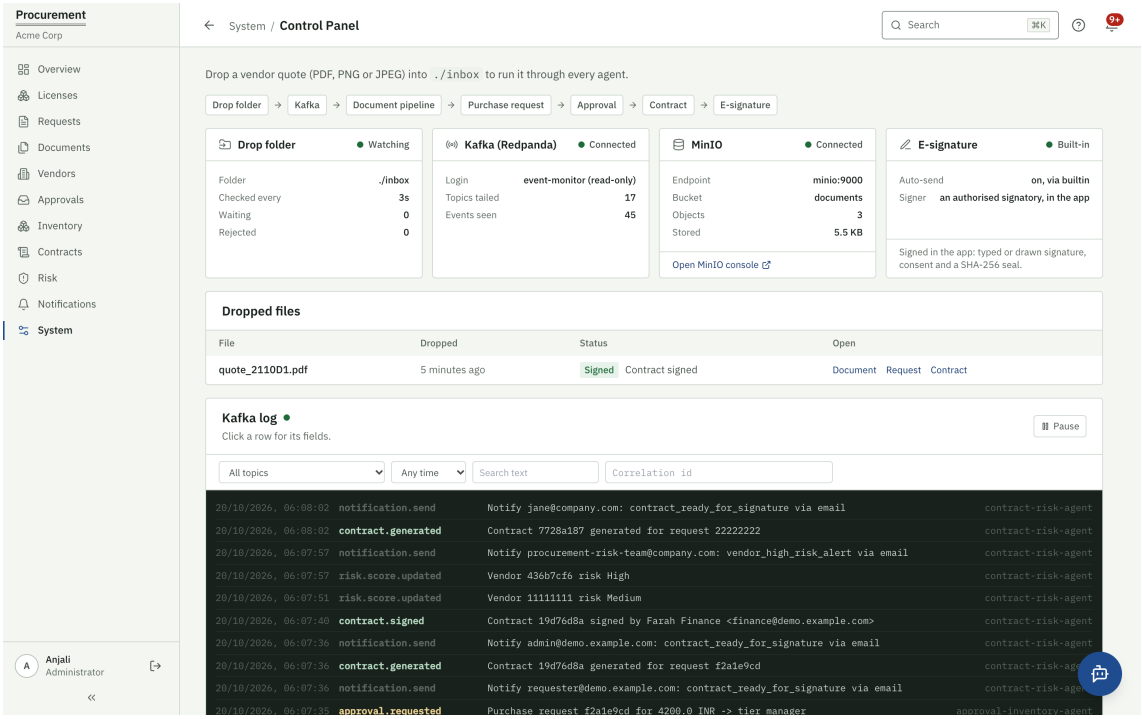


Figure 5.6: Control Panel: the drop-folder path from file to signature, the status of the drop folder, Kafka, MinIO and e-signature, a dropped quote that reached a signed contract unattended, and the live Kafka event log

where F excludes line items when a document has none, and θ is 0.8 or a per-vendor threshold learned from reviewers' corrections. Listing 5.1 shows the implementation.

```
def build_confidence_scores(classification_confidence, field_confidences, text_quality):
    scores = {"classification": round(classification_confidence * text_quality, 3)}
    for field_name, conf in field_confidences.items():
        scores[field_name] = round(conf * text_quality, 3)
    return scores

WEIGHTS = {
    "total": 0.30, "vendor_name": 0.20, "classification": 0.15,
    "document_number": 0.15, "document_date": 0.10, "line_items": 0.10,
}
OPTIONAL = {"line_items"}

def overall_confidence(confidence_scores):
    scores = {k: v for k, v in confidence_scores.items() if not (k in OPTIONAL and not v)}
    if not scores:
        return 0.0
    weights = {k: WEIGHTS.get(k, 0.1) for k in scores}
    return round(sum(scores[k] * w for k, w in weights.items()) / sum(weights.values()), 3)

def needs_review(overall, threshold=None):
    return overall < (load_threshold() if threshold is None else threshold)
```

Listing 5.1: Confidence scoring and the review gate (document-vendor-agent, `confidence.py`)

Race-safe stock reservation. A short-lived Redis lock per SKU guards the check-and-decrement, while the reservation itself is stored in PostgreSQL. Each acquisition uses a fresh token so that a retry can never release another request's lock (Listing 5.2).

```
async def reserve_stock(db, redis_client, sku, request_id, quantity) -> bool:
    lock = InventoryLock(redis_client, ttl=10)  # 10s safety TTL, released in ms
    lock_token = str(uuid.uuid4())              # unique per acquisition
```

```
    acquired = await lock.acquire(sku, lock_token)
    if not acquired:
        return False
    try:
        # Re-check availability inside the lock
        item = (await db.execute(select(Inventory).where(Inventory.sku == sku))).scalar_one_or_none()
        if not item or item.available_quantity < quantity:
            return False
        item.available_quantity -= quantity
        item.reserved_quantity += quantity
        await db.commit()
        return True
    finally:
        await lock.release(sku, lock_token)     # always release, even on failure
```

Listing 5.2: Stock reservation under a Redis lock (approval-inventory-agent, `inventory_service.py`)

Verified, replay-safe signature callback. The webhook is not JWT-authenticated, because the provider has no platform account. It is trusted only if an HMAC-SHA256 signature over the canonical JSON body matches, compared in constant time, and a provider event ID is accepted only once (Listing 5.3).

```
def verify_signature(payload: dict, signature: str) -> None:
    canonical = json.dumps(payload, sort_keys=True, separators=(",", ":")).encode("utf-8")
    expected = hmac.new(settings.esign_webhook_secret.encode("utf-8"), canonical, hashlib.sha256).
     hexdigest()
    if not hmac.compare_digest(expected, signature):
        raise WebhookSignatureError("webhook signature verification failed")

async def mark_contract_signed(db, kafka_producer, contract_id, signed_by, signed_at, provider_event_id,
      ...):
    existing = await db.get(ProcessedWebhookEvent, provider_event_id)
    if existing is not None:
        raise WebhookReplayError(f"webhook event {provider_event_id} already processed")
    contract = await db.get(Contract, contract_id)
    ...
    contract.status = "signed"
    db.add(ProcessedWebhookEvent(...))   # recorded in the same transaction as the status change
```

Listing 5.3: HMAC verification and replay protection (contract-risk-agent, `webhook_service.py`)

5.3 Testing

Testing was carried out at three levels: unit tests for each service and for the shared code, end-to-end tests through the gateway against the running system, and black-box probes of externally visible behaviour. Every defect fixed received a regression test, and each such test was checked by reverting the fix and confirming that it fails.

5.3.1 Unit Testing

Each service has its own pytest suite, run inside that service's Docker image with the source mounted (`./scripts/test-service.sh all`). The suites test logic, schemas, API behaviour and the wiring of every enforced business rule (each rule is set to two values and the test checks that behaviour changes). On 19 October 2026, after the dependency upgrades described in Section 5.3.4, all 844 tests passed, with none failing or skipped (Table 5.1). A further suite tests the shared code and configuration: the event outbox and dead-letter handling, the request state machine, Kafka permissions against the code,

log redaction, correlation IDs, e-mail approval actions, the business-rules registry and the refusal of published secrets in production mode. It passed 158 tests; 4 tests of the assistant's live script were skipped because they need a running language model.

Table 5.1: Unit tests per service (run on 19 October 2026)

Service	Passed	Areas covered
document-vendor-agent	334	Upload checks and ClamAV, e-mail and drop-folder intake, OCR runtime and preprocessing, Docling extraction, classification and classifier routing, field extraction and schema, LayoutLMv3 cross-check, confidence, vendor matching and onboarding, GSTIN and IFSC, payment quarantine, learning from corrections, pipeline resilience, event schemas, rules wiring, document access, the risk-controlled review gate
approval-inventory-agent	179	Spend tiers, approval authority and reminders, lifecycle enforcement, Redis lock and inventory reservation, reorder, invoice ledger, licence-usage anomaly model and reclaim, outbox publishing, order monitor, audit regressions, rules wiring, approval security, internal-token configuration
contract-risk-agent	198	Clause extraction and model routing, contract documents and access, in-app signing and its risk check, e-sign webhook and DocuSign client, renewal schedule, risk model and signals, sanctions screening, reminders, rules wiring, security configuration
notification-agent	58	Digest versus urgent routing, template rendering with strict undefined checks, consumer retry and dead-letter, notification-log access, event schemas
auth-service	75	Password hashing, JWT, session cookies and CSRF, account flows and refresh-token revocation (with a throwaway database), lockout and hardening, business rules, user profiles, demo-seed guard
Total	**844**	

5.3.2 Integration Testing

The end-to-end suites drive the running stack through the gateway exactly as a client would, without database shortcuts (Table 5.2). The gateway suite (`make e2e`) signs in as three roles, checks that ClamAV rejects the EICAR test file and accepts a clean PDF, creates a manager-tier request, approves it, generates and sends a contract, simulates the provider's signed callback with a real HMAC signature, confirms the signature, recomputes the vendor's risk, confirms that a notification reached Mailpit, and changes a business rule mid-test to check its history and reset. The invoice-lifecycle suite takes one fresh vendor's quote and invoice through all five services, and the document-pipeline suite submits normal, very large, corrupt, empty, wrongly typed and pathological files and simulates outages.

Integration exposed defects that unit tests could not, which were fixed in the shared configuration: the gateway did not strip the API prefix (every route returned 404), the broker advertised an unreachable address, UUID columns were typed as strings, Alembic version tables of two services collided, and ClamAV's slow start blocked the services that depended on it.

Table 5.2: End-to-end test suites

Suite	What it exercises	Result	Date run
`make e2e` (gateway)	Request, approval, contract, signature, risk, notification, rule change	17 / 17	19 Oct 2026
`invoice_lifecycle.py`	One vendor's quote and invoice through all five services	71 / 71	19 Oct 2026
`document_pipeline.py --quick`	Normal, oversized, corrupt, empty and mistyped files; duplicates; outages	48 / 52[a]	19 Oct 2026
`auth_flows.py`	Sign-up, e-mail confirmation, password change and reset, expired and reused links, lockout, rate limits	42 / 42	17 Oct 2026
`ui_flows.py` (Playwright)	Browser flows: lifecycle steps, approve and sign from the request page, inbox routing	11 / 11	16 Oct 2026

[a] The four failing checks concern one case, a 24 MB scan meant to exceed the 120-second extraction limit. Since the upgrade to Docling 2.140 it is read within the limit, so the case no longer exercises the time-out; the time-out itself is covered by a unit test.

5.3.3 Black-box Testing

Black-box probes test externally visible behaviour without reference to the code. The cases in Table 5.3 were sent through the gateway to the running system; the last column records the behaviour observed before a defect was fixed, where a probe first failed.

5.3.4 Security and Dependency Checks

Every service image was audited for known vulnerabilities with `pip-audit`, and the frontend with `npm audit`. Upgrading the web framework (FastAPI 0.148, Starlette 1.9) and the libraries that parse uploaded files (Pillow 12.3, pdfminer.six, Docling 2.140) reduced the findings from 302 to 152; the remainder are in the model-tracking server, which is not exposed, in parts of the OCR engine that the system does not call, and in test tooling. Started in production mode with the published default secrets, every service refuses to start; with new secrets, every service and worker starts (BB13). A full backup of the database and document store was restored into separate containers with identical record counts.

5.4 Evaluation of the Models

Each model was compared with a simple baseline on data it was not trained on, and each decision rule for keeping a model was fixed before the comparison was run. All training and test data is synthetic except the CUAD contracts, so these results compare methods on controlled data; they do not predict accuracy on an organisation's real documents or vendors.

5.4.1 Document Classification

The classifier was trained on 861 of 1,260 generated documents (clean, OCR-noise, missing-title and *other* paperwork), with 252 for validation and 252 for testing. A

Table 5.3: Black-box test cases (run through the gateway, 17–19 October 2026)

ID	Scenario	Expected and observed result	Before the fix
BB1	Full gateway flow (`make e2e`)	All 17 checks pass	–
BB2	Upload the EICAR test file; then a clean PDF	422 rejected; 201 accepted	–
BB3	Protected endpoint without a token	401 `missing bearer token`	–
BB4	Requester verifies a bank-detail change	403: only finance or admin may verify	–
BB5	Requester opens the approval inbox; approver opens it	403 for the requester; 200 for the approver	–
BB6	Signature webhook with a wrong HMAC signature	401 `invalid webhook signature`; contract unchanged	–
BB7	Malformed UUID in a path (`/vendors/2.1/risk`)	400 in the standard error envelope	–
BB8	Purchase request with amount `NaN`	422 validation error; nothing stored	Row stored; request list broken for all users
BB9	Hardware item with quantity −3	422 validation error	Stock corrupted (available 8, reserved −3)
BB10	Requester downloads another user's contract	404	PDF downloaded
BB11	Wrong password for a real account, and any password for an unknown address	Identical 401 `Invalid email or password.` (no account enumeration)	–
BB12	Refresh with a copy of the session cookie taken before logout	401 `session has ended`	200: new access token issued
BB13	Start each service in production mode with the published default secrets	Refuses to start, naming the secret	–

second test trained on 870 documents without one invoice layout and two kinds of *other* paperwork, and tested on 390 documents of exactly those kinds. Table 5.4 shows that the shipped router (the model when it is at least 60% confident, otherwise keywords) classified every test document correctly, against a macro-F1 of 0.912 and 0.844 for keywords. The keyword classifier was weakest on documents with no title (0.847 and 0.602). Because both test sets share the generator's vocabulary, perfect scores here mean the model learned these templates, not that it would be perfect on real documents.

Table 5.4: Document classification on synthetic test sets (accuracy and macro-F1)

Classifier	**Same templates (252 docs)**		**Held-out kinds (390 docs)**	
	Accuracy	Macro-F1	Accuracy	Macro-F1
Keyword classifier	0.929	0.912	0.854	0.844
Shipped: TF-IDF + logistic regression, keywords below 0.6	1.000	1.000	1.000	1.000

5.4.2 Field Extraction

Extraction was measured against ground truth on 15 generated PDFs and 6 generated images, each image also tested as rotated (4°), noisy and half-resolution copies (24 image variants). Every field of every PDF was correct, but only 47% of image fields without preprocessing and 49% with deskewing and upscaling (Table 5.5). Preprocessing helped on original and rotated images (47% to 53%) but hurt on noisy ones (47% to 44%). Image extraction is therefore the weakest part of the pipeline; this is why low-confidence documents go to review, and why OCR quality discounts every field's confidence (Equation 5.1).

Table 5.5: Field-extraction accuracy by field (share of documents with the field correct)

Input	**Type**	**Vendor**	**Number**	**Date**	**Total**	**Currency**	**All**
PDFs (15)	100%	100%	100%	100%	100%	100%	100%
Images, no preprocessing (24)	50%	33%	50%	42%	42%	67%	47%
Images, with preprocessing (24)	50%	54%	50%	38%	38%	67%	49%

5.4.3 Clause Extraction

The Legal-BERT model and the regular-expression rules were compared on the 102 contracts of CUAD's test split, real commercial contracts annotated by lawyers; a clause counts as found when the returned passage overlaps an annotated span. The model beat the rules on all three clause types (Figure 5.7), most clearly on expiration dates, which in CUAD are usually written relative to another date ("five years from the Effective Date") and so defeat the date rules. On 60 contracts generated from the project's own templates, both methods read every value correctly. The model was kept as the primary extractor under a rule fixed in advance (beat the rules on at least two of the three clauses and read every template value). Its cost is speed: a median of 17 seconds per contract on a CPU.

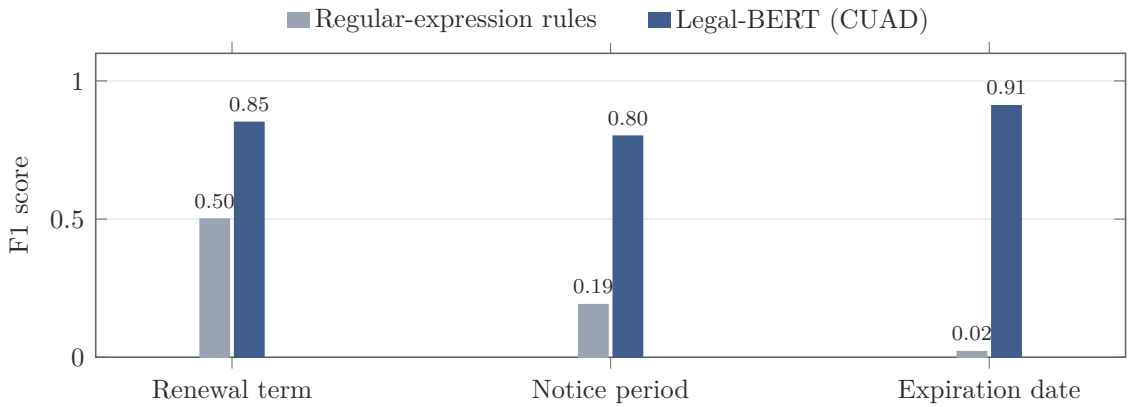


Figure 5.7: Clause extraction F1 on the CUAD test split (102 contracts; 26, 16 and 78 of them contain the respective clause)

5.4.4 Licence-Usage Anomaly Detection

The anomaly model was evaluated on 600 simulated licences (seed 2026), 30% of which follow an anomalous pattern (gradual decline, sudden drop-off or abandonment) while the rest are steady, have weekend dips or are stable at low use. The licences were split 360 / 120 / 120 for training, validation and testing. Figure 5.8 compares the shipped model with three alternatives on the test set. Scale-free features (utilisation and retention ratios rather than raw seat counts) raised the ROC-AUC from 0.849 to 0.954, and both are far above the earlier model trained on only five licences (0.659) and above ranking by low utilisation alone (0.687), which is what a flat inactivity rule does. The thresholds were chosen on the validation set: at the "anomalous" threshold (0.39) the test precision was 0.83 and the recall 0.69; at the "watch" threshold (0.34) they were 0.79 and 0.83. Abandoned licences were always flagged and sudden drop-offs 87% of the time, but gradual declines only 36% of the time, while 4–10% of the normal licences crossed the anomalous threshold. The grace period and human approval exist for exactly these false positives.

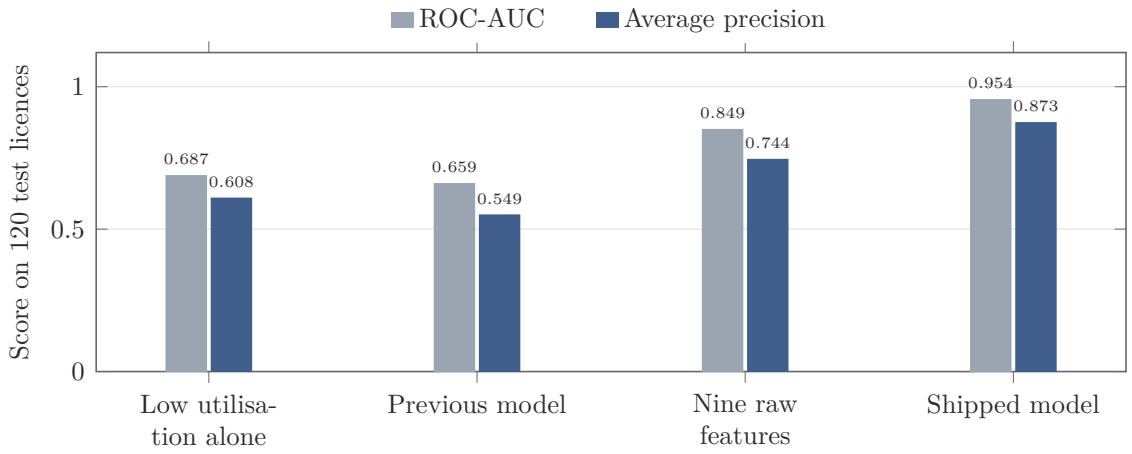


Figure 5.8: Licence-anomaly detection on 120 simulated test licences. "Previous model": original features trained on 5 licences; "Nine raw features": original features trained on the same 360 licences; "Shipped model": seven scale-free features

5.4.5 Vendor Risk Scoring

The risk model was evaluated on 2,000 synthetic vendors (360 with an incident), split 1,600 / 400. The incident labels were drawn from a known formula of the six features plus random noise, so even a perfect model cannot exceed the ROC-AUC of that formula, 0.739. The Random Forest reached 0.715 (0.726 with 20% of the values missing), close to that ceiling, with an accuracy of 0.72 at a threshold of 0.5. More importantly for its use, the bands are ordered by real incident rate (Figure 5.9): 6.5% of Low-band vendors, 20% of Medium and 40.3% of High had an incident. The most important features were on-time delivery (0.349), financial stability (0.210) and tenure (0.207). Twelve hand-written red-team profiles, including a perfect vendor with a cancelled GSTIN (forced High by the override), a vendor with no data on file (Medium, never High) and vendors with partial data, all received the expected band.

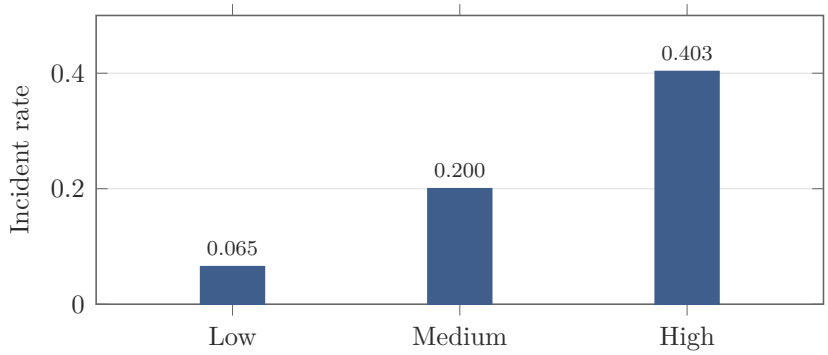


Figure 5.9: Incident rate per risk band on 400 synthetic test vendors (153 Low, 185 Medium, 62 High)

5.4.6 Calibrating the Document Review Gate

The review gate of Equation 5.1 sends a document to a person when its weighted confidence is below 0.8; how many errors that threshold lets through was unknown. A controlled experiment measured it and asked whether evidence from the organisation's records—the vendor master, the open purchase orders and earlier invoices—allows more documents to skip review at a fixed error rate.

Data. A seeded generator produced a vendor master of 100 vendors (each with a GSTIN, bank details and a currency, with invoice volume following a Zipf-like distribution), 1,649 purchase orders and 3,000 invoices against them. Of the invoices, 29.8% carry one injected business error: a unit price or quantity above the order, a resubmitted earlier invoice, a look-alike vendor name, changed bank details, a reference to another vendor's order, a wrong GSTIN, or a total that does not equal subtotal plus tax. The invoices were rendered as text PDFs (55%) or as clean, mildly degraded and heavily degraded scans, and were read with the production extraction code. An invoice counts as *correct* when its vendor, total, currency and invoice number were read correctly and it carries no business error; 59.9% of invoices are correct.

Systems. B0 is the current rule (confidence ≥ 0.8 and no forced-review flag). B2 accepts an invoice when every record check passes (vendor matched, order matched, no

duplicate, bank details and GSTIN as on file). B1 and P are calibrated gates: a logistic-regression or gradient-boosting model estimates the probability that an invoice is correct, from document evidence only (B1) or from document and record evidence (P), and the acceptance threshold is chosen with Learn-then-Test [24] so that the error rate among accepted invoices is at most α with probability at least $1 - \delta$ ($\delta = 0.1$). All systems were evaluated on 20 random splits in which the training, calibration and test sets contain different vendors.

Table 5.6: Review-gate systems on 3,000 synthetic invoices (means over 20 vendor-disjoint splits). Coverage: share of test invoices accepted without review; risk: share of accepted invoices that are wrong; violations: share of splits in which risk exceeded α

System	$\alpha = 2\%$		$\alpha = 5\%$		**AUROC**
	Coverage	Risk	Coverage	Risk	
B0: current confidence rule	41.2%	6.8%	41.2%	6.8%	–
B2: all record checks pass	58.9%	12.8%	58.9%	12.8%	–
B1: calibrated, document evidence	0.0%	–	0.0%	–	0.741
P: calibrated, document and record evidence	0.0%	–	1.7%	0.2%	0.947
P with a number-independent duplicate check[b]	12.1%	0.3%	35.4%	2.3%	0.971
B0 and B2 are fixed rules, so their figures do not depend on α; they violate both budgets in 90–100% of splits. P is shown with gradient boosting; AUROC is threshold-free. [b] Exploratory, defined after the diagnosis below; violation rate 5% at $\alpha = 2\%$ and 10% at $\alpha = 5\%$.					

Results. Table 5.6 gives the main result. The current rule accepts 41% of invoices, but 6.8% of the accepted ones are wrong, and accepting whenever every record check passes is worse (12.8%). Record evidence improves the ranking of invoices substantially: the area under the ROC curve rises from 0.74 with document evidence alone to 0.95 with record evidence, and removing the purchase-order match, the largest contributor, lowers it to 0.83. Nevertheless, no calibrated gate could certify an error budget of 2% or 5% with a useful share of invoices: among the best-ranked invoices about 5% remain wrong, and Learn-then-Test correctly refuses to accept what it cannot certify. Inspection of these invoices showed that most were resubmitted duplicates with a new invoice number. The production duplicate check requires the two numbers to be at least 85% similar, so it misses this case, and the order still has quantity left, so the three-way match passes too. Adding a duplicate check that ignores the number—equivalent to setting one business rule to zero—lets 12% of invoices through at a 2% budget and 35% at a 5% budget.

Limitations. The invoices, records and error rates are synthetic, from a single generator. The second gate was defined after inspecting the results and must be confirmed on a new seed. Vendor-disjoint splits violate the exchangeability that Learn-then-Test assumes, which may explain violation rates above δ in some settings. External validity on public invoice datasets has not been tested.

5.5 Discussion

The results support the design in three ways. First, the agents work together as intended: a request or document passes through all five services without manual steps, and the outbox, retries and state machine keep each step consistent when events arrive late or twice. Second, each trained model earned its place against a simple alternative on held-out data under a decision rule fixed in advance: the classifier and Legal-BERT replaced rules only because they beat them, and the licence model was redesigned after its first version separated the usage patterns poorly. Third, the controls that matter most in a financial process—fail-closed scanning, dual control of bank details, role checks, replay protection, session revocation and refusal of default secrets—were verified by black-box probes against the running system.

The results also expose the system's limits. Image extraction is weak (49% of fields correct), so scanned documents depend on review. The licence model misses most gradual declines and flags some normal licences, and a retrained model moved one demonstration licence just below its threshold; its proposals must therefore stay subject to approval. The classifier and risk-model figures come from synthetic data whose structure the models can learn; only Legal-BERT was evaluated on real documents, and those are US commercial contracts. Finally, the review-gate experiment shows that the confidence rule in use lets through 6.8% wrong invoices among those it accepts, more than any of the error budgets studied, and that the limiting factor is a gap in one check rather than in the models: record evidence makes risk-controlled automation possible only once duplicate resubmissions are caught.

Chapter 6

Conclusion & Future Scope

This report presented the design, implementation and evaluation of an IT Procurement Intelligence Platform. Document extraction, clause analysis, approval workflows, anomaly detection and risk scoring are usually treated as separate problems; the platform joins them in one event-driven system of four cooperating agents—Document and Vendor Intelligence, Approval and Inventory Intelligence, Contract and Risk Analysis, and a Platform agent.

In the implemented system, a document arriving by upload, e-mail or a watched folder is scanned, read, classified, matched to a vendor and confidence-scored; a purchase request is routed through spend-tier approvals with deadlines, escalation and race-safe stock reservation; an approved request becomes a contract that is signed in the application after the vendor's risk is shown, read by a legal language model and tracked for renewal; under-used licences are proposed for reclaim with explanations and a grace period; and every vendor carries an explainable risk score with hard overrides and drift monitoring. The system passes 844 service unit tests and end-to-end tests through its gateway.

Three findings stand out. Trained models were adopted only where they beat a simple baseline on held-out data. The security controls essential to a payment process were verified against the running system rather than assumed. And the evaluation of the review gate showed that the heuristic confidence rule accepts 41% of invoices with 6.8% of them wrong; that evidence from the organisation's own records markedly improves the ranking of documents (AUROC 0.74 to 0.95); and that a risk-controlled gate becomes feasible only once the duplicate check catches resubmitted invoices.

Limitations.

- All training and test data except the CUAD contracts is synthetic; no accuracy claim is made for an organisation's real documents, licences or vendors.
- Field extraction from scanned images is weak (49% of fields correct), so such documents depend on human review.
- The licence-anomaly model misses most gradual declines and flags a few normal licences; its proposals always require approval.
- The review gate in production still uses the uncalibrated threshold of 0.8, and the duplicate check misses resubmitted invoices that carry a new number.
- Adverse-media screening and budget tracking are not built, and live GSTIN registry lookups are disabled by default.
- Signing happens in the application; an external e-signature provider would need the same risk check at send time.
- The system runs on a single Docker host with unencrypted Kafka traffic, and the test suites are not yet run by the CI pipeline.

Future scope.

- Fix the duplicate check, repeat the review-gate experiment on a new seed and on public invoice data, and switch the production gate to the calibrated one behind a business rule.

- Evaluate and improve extraction on real, anonymised procurement documents, for example by fine-tuning LayoutLMv3 on labelled invoices, and retrain the classifier from reviewers' corrections.
- Close the main gaps with commercial suites (Section 2.4): budget control before approval, ERP integration, a supplier portal and sourcing events.
- Add adverse-media screening, GST return reconciliation and live GSTIN registry checks with quota management.
- Connect an external e-signature provider with the risk acknowledgement enforced at send time.
- Encrypt Kafka traffic, run the test suites in CI, and deploy to a managed environment to measure throughput and latency under load.

References

[1] R. Smith, "An overview of the Tesseract OCR engine," in *Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR)*, 2007, pp. 629–633. doi:10.1109/ICDAR.2007.4376991

[2] C. Cui *et al.*, "PaddleOCR 3.0 technical report," arXiv:2507.05595, 2025. arxiv.org/abs/2507.05595

[3] C. Auer *et al.*, "Docling technical report," arXiv:2408.09869, 2024. arxiv.org/abs/2408.09869

[4] Y. Xu, M. Li, L. Cui, S. Huang, F. Wei, and M. Zhou, "LayoutLM: Pre-training of text and layout for document image understanding," in *Proc. 26th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining*, 2020, pp. 1192–1200. doi:10.1145/3394486.3403172

[5] Y. Huang, T. Lv, L. Cui, Y. Lu, and F. Wei, "LayoutLMv3: Pre-training for document AI with unified text and image masking," in *Proc. 30th ACM Int. Conf. Multimedia*, 2022, pp. 4083–4091. doi:10.1145/3503161.3548112

[6] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," in *Proc. NetDB Workshop*, 2011. [Online]

[7] C. Richardson, *Microservices Patterns: With Examples in Java*. Shelter Island, NY, USA: Manning, 2018. manning.com/books/microservices-patterns

[8] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 413–422. doi:10.1109/ICDM.2008.17

[9] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017. [Online]

[10] I. Chalkidis, M. Fergadiotis, P. Malakasiotis, N. Aletras, and I. Androutsopoulos, "LEGAL-BERT: The muppets straight out of law school," in *Findings of the Association for Computational Linguistics: EMNLP*, 2020, pp. 2898–2904. doi:10.18653/v1/2020.findings-emnlp.261

[11] D. Hendrycks, C. Burns, A. Chen, and S. Ball, "CUAD: An expert-annotated NLP dataset for legal contract review," in *Proc. NeurIPS Datasets and Benchmarks Track*, 2021. arxiv.org/abs/2103.06268

[12] D. Araci, "FinBERT: Financial sentiment analysis with pre-trained language models," arXiv:1908.10063, 2019. arxiv.org/abs/1908.10063

[13] `alex-apostolo/legal-bert-base-cuad`, LEGAL-BERT fine-tuned on CUAD, Hugging Face model repository. huggingface.co/alex-apostolo/legal-bert-base-cuad (accessed Oct. 2026).

[14] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001. doi:10.1023/A:1010933404324

[15] B. Yurdakul, "Statistical properties of population stability index," Ph.D. dissertation, Dept. of Statistics, Western Michigan University, 2018. scholarworks.wmich.edu/dissertations/3208

[16] OpenSanctions, "Consolidated sanctions targets," dataset, CC BY-NC 4.0. opensanctions.org/datasets/sanctions (accessed Oct. 2026).

[17] Q. Wu *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," arXiv:2308.08155, 2023. arxiv.org/abs/2308.08155

[18] Scry AI, "Best procurement automation software," blog article, 2026. scryai.com/blog/best-procurement-automation-software (accessed Oct. 2026).

[19] Coupa Software, "The agentic enterprise: September 2026 release," product blog, 2026. coupa.com/blog/agentic-enterprise-september-2026-release (accessed Oct. 2026).

[20] Coupa Software, "Coupa for digitizing procurement, invoicing, and payments," product demonstration page. coupa.com/demo (accessed Oct. 2026).

[21] Zapro, "Procurement and vendor management platform," product website. zapro.ai (accessed Oct. 2026).

[22] SAP SE, "Next-generation SAP Ariba: smart source-to-pay procurement software," product page. sap.com/products/spend-management (accessed Oct. 2026).

[23] TYASuite, "Procurement software," product page. tyasuite.com/procurement-software (accessed Oct. 2026).

[24] A. N. Angelopoulos, S. Bates, E. J. Candès, M. I. Jordan, and L. Lei, "Learn then test: Calibrating predictive algorithms to achieve risk control," arXiv:2110.01052, 2021. arxiv.org/abs/2110.01052

[25] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. jmlr.org/papers/v12/pedregosa11a.html

[26] A. Biryukov, D. Dinu, and D. Khovratovich, "Argon2: New generation of memory-hard functions for password hashing and other applications," in *Proc. IEEE European Symp. Security and Privacy (EuroS&P)*, 2016, pp. 292–302. doi:10.1109/EuroSP.2016.31

Acknowledgement

The success of a project like this, which demands technical expertise, patience and generous support from guides, is possible only when team members work together. We take this opportunity to express our gratitude to those who were instrumental in its completion. We thank our project guide **Dr. Jyoti More** and our external guide **Mr. Alok** for their tremendous support and help; without them, this project would not have reached fruition. We also thank our project coordinator **Dr. Rakhi Kalantri** for regular inputs on documentation and the project timeline, and our Head of Department **Dr. M. Kiruthika** for her encouragement. Finally, we thank our Principal, **Dr. S. M. Khot**, and our college, **Fr. C. Rodrigues Institute of Technology, Vashi**, for the opportunity and the environment to learn and grow.

Project Group Members:

1. More Anjali Dinesh, 1023207

_____

2. Sankaye Vaidehi Jagdeep, 1023239

_____

3. Sawant Niraj Ravindra, 1023240

_____

4. Shete Anooj, 1023250

_____

Appendix A: Timeline Chart

Figure A.1 shows the project timeline. The bars up to October 2026 are taken from the dates of the commits in the project repository (113 commits between 4 July and 20 October 2026) and from the dated review and audit records; the bars for November and December are the planned testing and submission phases.

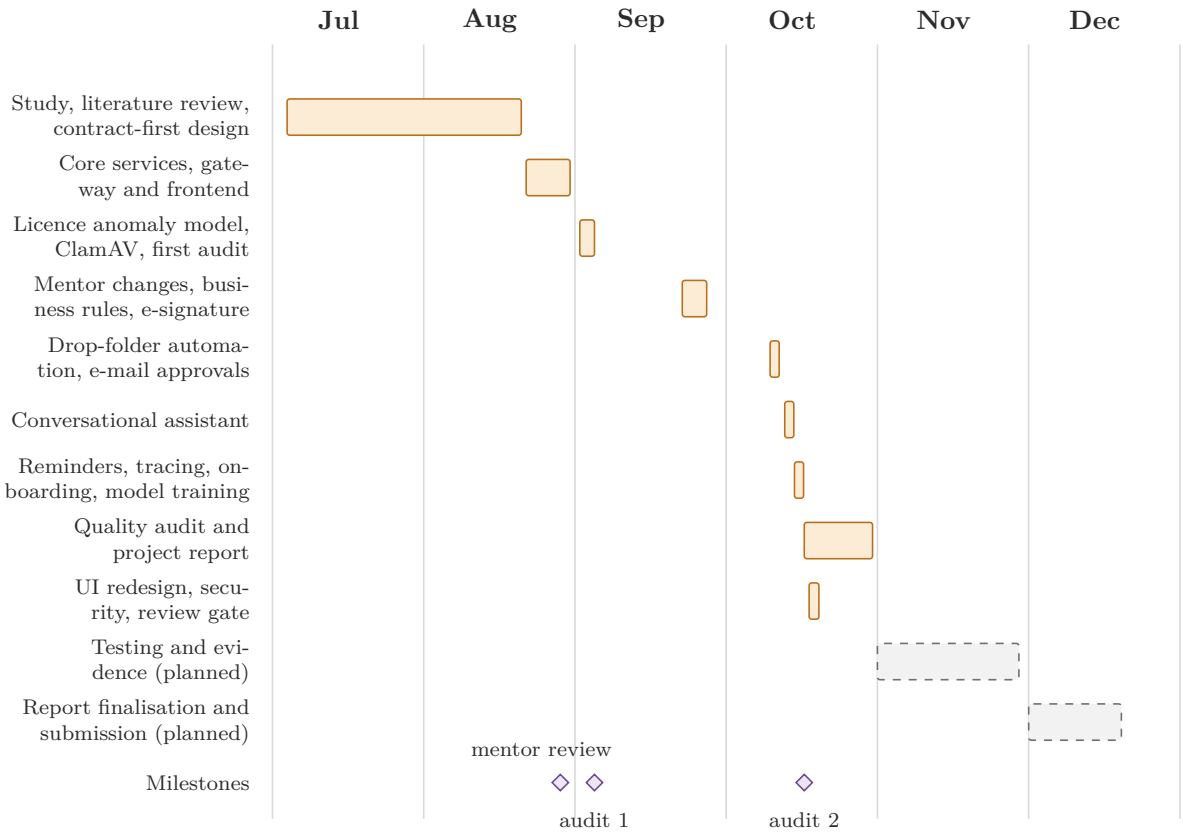


Figure A.1: Project timeline (solid: completed, from the repository history; dashed: planned)

Appendix B

The project team has identified the following Sustainable Development Goals (SDGs) as relevant to the project. The mapping shows the relationship between the project problem, the proposed solution and each selected SDG.

SDG Mapping of the Project

Sr. No.	**SDG No.**	**Complete Name of SDG**	**Rationale / Contribution of the Project**
1	9	Industry, Innovation and Infrastructure	The platform applies machine learning, event-driven software architecture and workflow automation to a core enterprise process, improving the technological capability and efficiency of IT procurement.
2	12	Responsible Consumption and Production	Duplicate-invoice detection, inventory reservation and reorder suggestions, and the reclaim of under-used software licences reduce wasted purchases and idle resources.
3	16	Peace, Justice and Strong Institutions	Role-based access, separation of duties in approvals, dual control of payment-detail changes, sanctions screening, audit logging and explainable decisions support accountable, transparent and fraud-resistant institutions.

_____

Signature of Project Guide

Name of Project Guide: Dr. Jyoti More

Date: _____

Appendix C

Appendix C: Complex Engineering Problem Identification and Evaluation

The Washington Accord emphasises the ability of engineering graduates to identify, analyse and solve complex engineering problems using appropriate engineering knowledge, modern tools, multidisciplinary thinking and analytical reasoning. The Major Project provides an opportunity for students to demonstrate these abilities through a structured and evidence-based project.

This appendix provides a structured mechanism to assess the complexity of the Major Project. The assessment is first carried out by the student team and then verified by the Project Guide.

A. Project Details

Project Title	Agentic AI Procurement Solution
Group / Team No.	
Name of Student(s)	More Anjali Dinesh (1023207), Sankaye Vaidehi Jagdeep (1023239), Sawant Niraj Ravindra (1023240), Shete Anooj (1023250)
Project Guide	Dr. Jyoti More
Academic Year	2026–2027

B. Complexity Evaluation

For each parameter, the student team assigns a score from **0 to 3** with a justification or evidence. The Project Guide reviews the assessment and records the verified score in the final column.

Sr. No.	Complexity Parameter	Student Score	Student Justification / Evidence	Guide Score
1	Problem novelty and research depth	2	Integrates document AI, workflows and risk scoring in one governed pipeline; the individual techniques are established (Chapter 2).	
2	Multidisciplinary knowledge requirement	3	Combines document processing, machine learning, natural language processing, distributed systems, workflow engines, security and web development.	
3	System design complexity	3	Six FastAPI services and three workers in 24 running containers, a shared event and database contract, a transactional outbox, durable workflows and a gateway.	
4	Technology integration	3	FastAPI, Redpanda, Temporal, PostgreSQL, Redis, MinIO, ClamAV, MLflow, Prometheus, Grafana, Loki, Ollama and React in one containerised stack.	
5	Data analysis and experimentation	2	Three models trained and kept only after beating a baseline on held-out data; Legal-BERT evaluated on 102 CUAD test contracts; extraction measured on 15 PDFs and 24 image variants. All training data is synthetic (Section 5.4).	
6	Societal / industrial relevance	2	Addresses duplicate payments, unused licences and payment-fraud risks that affect organisations.	
7	Stakeholder involvement	2	Four user roles modelled; guidance from an external industry guide and mentor feedback rounds.	
8	Sustainability considerations	1	Reduces wasted purchases and unused licences.	
	Total Score	**18 / 24**		**/ 24**

Scoring Guideline: 0 – Not applicable / negligible; 1 – Basic level of complexity; 2 – Moderate level of complexity; 3 – Significant / high level of complexity. Maximum score: 24.

C. Final Project Classification

The final classification is based on the score verified by the Project Guide.

Score Range	Project Classification
0–8	Simple Engineering Problem
9–16	Moderately Complex Engineering Problem
17–24	Complex Engineering Problem

Final Classification: ☐ Simple ☐ Moderately Complex ☐ Complex Engineering Problem

_____ Date: _____

Project Guide Signature

Appendix D: Project Outcomes

Publications and Competitions

Outcome	Details	Status
Research paper	A paper on calibrated, evidence-based review gates for procurement documents (provisional title: *Measuring the Value of Enterprise Context in Selective Automation of Procure-to-Pay Documents*)	First results obtained (Section 5.4.6); draft in preparation
Project competitions	To be added by the team	–

Technical Outcomes

- A working, containerised platform of six services and three workers that takes a procurement document or request from intake to a signed contract, receipt and invoice match.
- Three trained models (document classifier, licence-anomaly detector, vendor risk model) and one adopted language model (Legal-BERT for clauses), each kept only after beating a baseline on held-out data (Section 5.4).
- 844 service unit tests, a shared-code suite and end-to-end suites through the gateway, with a regression test for every defect fixed.
- A seeded procurement-world generator and evaluation harness for risk-controlled document automation.
- Documentation for reproduction: setup and run scripts, backup and restore scripts, a deployment checklist, a technical reference, a business-rules register and dated evaluation results.